OnBuch+

SIGNED COURSE • PREMIUM

COMPUTER SCIENCE • UPPER SIXTH | UPPER SIXTH — COMPUTER SCIENCE

CHAPTER 12

MANAGING INPUT/OUTPUT OPERATIONS AND FILE SYSTEMS

SUGGESTED TIME: 6 PERIODS

IN THIS CHAPTER

This chapter explains how the operating system manages input/output devices, transfers data between memory and peripherals, and organises files on mass storage. It also covers file access methods, disk management, and the security and error-handling mechanisms that keep a computer system reliable.

- By the end of this chapter I can describe the role of the I/O subsystem and device drivers in an operating system.
- By the end of this chapter I can distinguish programmed I/O, interrupt-driven I/O and DMA, and justify the choice of each mechanism.
- By the end of this chapter I can explain the functions of a file system (naming, allocation, directories, free-space management).
- By the end of this chapter I can compare contiguous, linked and indexed file allocation methods.
- By the end of this chapter I can describe the structure of mass storage devices and compute simple disk access times.
- By the end of this chapter I can compare sequential, direct and indexed-sequential file access methods with concrete examples.

DOWNLOAD ONBUCH
Free • Google Play
AI tutor, past papers, results

GO ONBUCH+
All signed courses, unlimited AI tutor, PDF sheets — OnBuch+ tab in the app

THIS SIGNED COURSE CONTAINS

COURSE complete, illustrated, step by step

METHODS and worked examples

EXERCISES exam style + solutions

SUMMARY SHEET the essentials to remember

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+

CONTENTS

1. I/O Device Management
2. I/O Transfer Mechanisms
3. File Systems and File Management
4. Mass Storage and File Access Methods
5. OS Security and Error Management
6. Integration activity
7. Methods and worked examples
8. Exercises
9. Solutions to the exercises
10. Summary sheet

CHAPTER OBJECTIVES

- By the end of this chapter I can describe the role of the I/O subsystem and device drivers in an operating system.
- By the end of this chapter I can distinguish programmed I/O, interrupt-driven I/O and DMA, and justify the choice of each mechanism.
- By the end of this chapter I can explain the functions of a file system (naming, allocation, directories, free-space management).
- By the end of this chapter I can compare contiguous, linked and indexed file allocation methods.
- By the end of this chapter I can describe the structure of mass storage devices and compute simple disk access times.
- By the end of this chapter I can compare sequential, direct and indexed-sequential file access methods with concrete examples.
- By the end of this chapter I can explain how the OS protects files and devices (permissions, authentication) and handles I/O errors.
- By the end of this chapter I can apply these concepts to analyse a real scenario such as a school computer room or a cybercafé network.

PREREQUISITES

- Basic computer hardware: CPU, memory, peripherals (Lower Sixth, hardware unit)
- Role and functions of an operating system (Lower Sixth)

- Number systems and units of data storage (bit, byte, KB, MB, GB)
- Basic notions of files, folders and extensions from practical computer use
- Concept of interrupts and program execution cycle (introduced in Lower Sixth)

1 I/O DEVICE MANAGEMENT

At a busy cybercafé in Buea, a student waits several minutes for a 200-page project to print while another copies photos to a flash drive that suddenly becomes "corrupted" and unreadable. Why does the computer slow down when many devices work at once? Who decides which program may use the printer first, and why can a file "disappear" after a power cut during copying? Behind every click, the operating system is silently managing input/output devices, organising files on disk, and protecting data from errors and unauthorised access. This chapter opens the door to that hidden world.

The **problem** we must solve is the following: a modern computer is connected to dozens of very different devices (keyboard, mouse, screen, printer, hard disk, flash drive, network card), each with its own speed, its own electronic signals and its own way of working. Yet the programmer who writes a simple statement such as `PRINT "Hello"` does not want to know anything about printer electronics. Someone must hide this complexity. That "someone" is the **I/O subsystem** of the operating system.

1.1 The Role of the I/O Subsystem

Observe a simple fact: a program written in C can read data from a keyboard, from a hard disk or from a network connection using almost the same instructions (`read`, `scanf`). The programmer never writes different code for a Logitech keyboard and a Dell keyboard. This is possible because the operating system places itself between the program and the hardware.

DEFINITION **I/O Subsystem**

The **I/O subsystem** (input/output subsystem) is the part of the operating system responsible for managing all communications between the computer's programs and its peripheral devices. Its main missions are to:

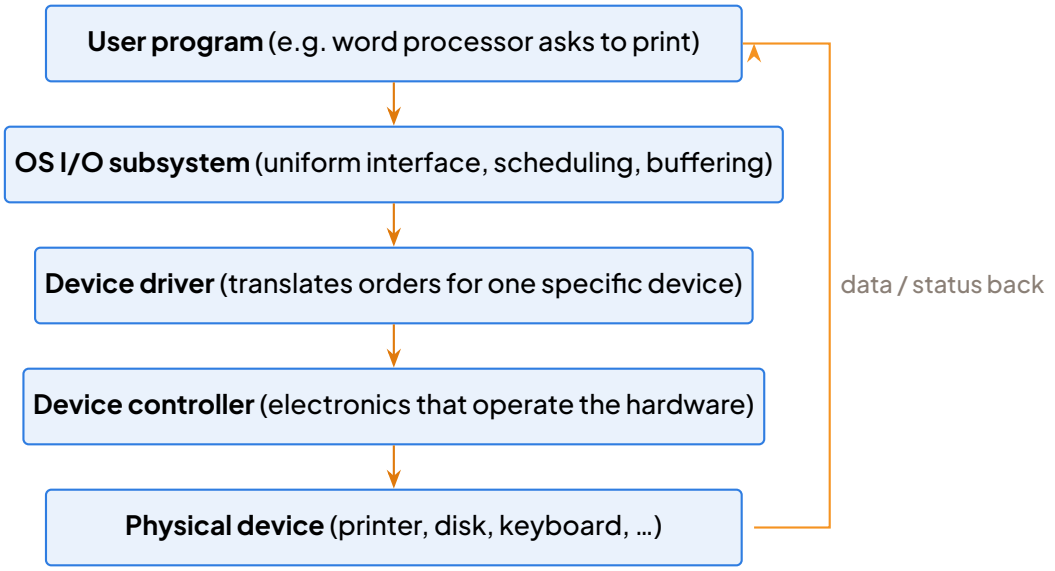
- hide hardware differences from users and programs, presenting every device through a simple, uniform interface (open, read, write, close);
- control and coordinate the devices so that several programs can share them without conflict;
- improve performance using techniques such as buffering, caching and spooling;
- detect and handle errors occurring during transfers (paper jam, disk full, power failure).

KEY POINTS **Key idea**

The I/O subsystem offers abstraction: the user sees a simple, uniform service ("read a character", "write a block"), while the OS alone deals with the messy, device-specific details. This is exactly like a hotel receptionist: guests make one simple request, and the

receptionist coordinates the cook, the cleaner and the driver behind the scenes.

The work is organised in **layers**, each layer speaking only to the layer directly above or below it.



Layered architecture of I/O management: each layer hides the complexity of the layers below it.

1.2 Categories of Devices

Before managing devices, the OS must classify them, because different kinds of devices need different treatment.

First classification: by direction of data flow.

- **Input devices** bring data into the computer: keyboard, mouse, scanner, microphone.
- **Output devices** send data out of the computer: monitor, printer, speakers, projector.
- **Storage devices** do both: data can be written to them and read back later: hard disk, SSD, flash drive, memory card, DVD.

Second classification: by unit of transfer.

DEFINITION **Character and block devices**

A **character device** transfers data one character (byte) at a time, as a stream, without any fixed-size structure. Examples: keyboard, mouse, serial port, printer. A **block device** transfers data in fixed-size blocks (typically 512 bytes or 4 KB), and each block has an address so it can be read or written independently, in any order. Examples: hard disk, SSD,

flash drive.

The distinction matters enormously: because blocks are addressable, block devices support **random access** (you can jump directly to block 5000), which is why file systems are built on them. A keyboard, by contrast, delivers a stream: you cannot ask it to "give back the third keystroke again".

METHOD **How to decide whether a device is character or block oriented**

Ask yourself the following questions, in order:

1. Unit of transfer: does the device exchange data byte by byte (stream) or in fixed-size chunks? Byte by byte ⇒ character device; fixed chunks ⇒ block device.
2. Addressability: can I ask for "piece number *n*" directly, in any order? Yes ⇒ block device; no (data arrives in sequence and is consumed) ⇒ character device.
3. Check with examples: keyboard, mouse, printer ⇒ character; hard disk, SSD, flash drive ⇒ block.

Third classification: by sharing.

- A **dedicated device** can be used by only one program at a time. A printer is the classic example: if two documents were sent to it truly simultaneously, their pages would interleave into nonsense.
- A **shared device** can serve several programs concurrently, because each request is short and independent. A hard disk is shared: the OS interleaves read/write requests from many programs, block by block, without confusion.

1.3 Device Controllers and I/O Ports

The CPU cannot plug a USB cable directly into its own circuits. Every peripheral is connected through a small electronic circuit called a **device controller** (or adapter): a disk controller, a USB controller, a graphics controller. The controller is the "translator and manager" of one device (or one family of devices): it receives commands from the CPU, operates the motors, heads or lasers of the device, and reports the device's status.

Communication happens through **I/O ports**: special registers of the controller that the CPU can read and write, rather like numbered mailboxes.

- The CPU writes a **command** into the command register (e.g. "read block 1200").
- It places or collects **data** in the data register.
- It reads the status register to know whether the device is **busy**, **ready**, or in **error**.

EXEMPLE **Reading a key pressed on the keyboard**

1. You press the letter `A`; the keyboard controller stores its code in its data register and sets the status register to “data ready”.
2. The controller signals the CPU (through an interrupt): “I have something for you.”
3. The CPU runs the keyboard driver, which reads the data register, converts the code into the character `A`, and delivers it to the waiting program.
4. The status register returns to “ready”, waiting for the next key.

All of this takes far less than a millisecond.

1.4 Device Drivers

The controller understands electrical signals; the OS understands general orders like “write this data”. Between the two sits a piece of software that knows one particular device perfectly.

DEFINITION **Device driver**

A **device driver** is a small program, part of the operating system, that controls one specific device (or model of device). It translates the general requests of the OS (“print this page”) into the exact sequence of commands understood by that device’s controller, and it translates the device’s signals back into information the OS can use.

This explains a familiar experience: when you buy a new printer in Douala and connect it to a Windows computer, the system may say “Installing device driver”. Windows searches its library (or the Internet, or the CD supplied by the manufacturer) for the driver matching that exact printer model. **Without the correct driver, the device simply cannot work**, even though it is physically connected, because the OS does not know how to “talk” to it. This is also why a very old printer may refuse to work on a brand-new computer: no compatible driver exists.

WARNING **Common mistake**

Do not confuse the driver (software, part of the OS) with the controller (hardware, an electronic circuit). The driver commands; the controller executes on the physical device. Exam questions often ask you to place each one in the layered architecture.

1.5 Device Independence and Uniform Naming

Thanks to drivers, the OS can offer **device independence**: programs are written against a general device interface, not against a specific model. Your word processor prints the same way whether the printer is an HP, a Canon or an Epson; only the driver changes.

To make this concrete, every OS gives devices **uniform names**:

- In **Linux**, devices appear as special files in the directory `/dev`: `/dev/sda` (first disk), `/dev/lp0` (first printer), `/dev/tty` (terminal). A program can "write to the printer" exactly as it would write to an ordinary file.
- In **Windows**, storage devices receive **drive letters** (`C:`, `D:`, `E:` for a flash drive), and printers are named objects managed by the print system.

The benefit is powerful: the same program logic works for any device, and a device can be replaced without rewriting any application.

1.6 Buffering, Caching and Spooling

Devices are slow compared with the CPU: a disk is thousands of times slower, a printer millions of times slower. To prevent the fast CPU from waiting idle for slow devices, the OS stores data temporarily in memory. Three techniques must be carefully distinguished.

DEFINITION **Buffering**

Buffering is the use of a temporary memory area, called a buffer, to hold data while it is being transferred between a program and a device, in order to absorb the speed difference between them. Data passes through the buffer once and is then gone.

Example: when you type quickly, the characters wait in a keyboard buffer until the program reads them; nothing is lost even if the program was momentarily busy.

DEFINITION **Caching**

Caching is the keeping of a copy of frequently used data in fast memory, so that future requests for the same data can be served quickly without returning to the slow device. The copy is kept deliberately and may be reused many times.

Example: after you open a file once, the OS keeps some of its blocks in a disk cache; opening it a second time is almost instant.

WARNING **Buffering ≠ caching**

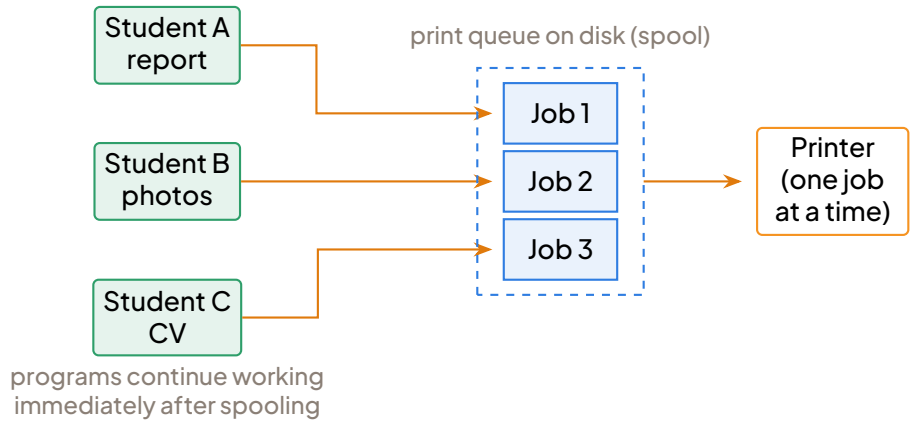
A very common exam mistake is to confuse the two. Buffering holds data in transit, used once, to smooth a transfer between a fast and a slow partner. Caching keeps a copy of data for reuse, to avoid repeating slow work. A buffer is a corridor through which data passes; a cache is a cupboard where favourite items are kept close at hand.

The third technique solves the problem of dedicated devices like printers.

DEFINITION **Spooling**

Spooling (Simultaneous Peripheral Operations On-Line) is a technique in which jobs intended for a dedicated device (typically a printer) are first written completely to a waiting area on disk (the spool or print queue), and are then sent to the device one after another by a system process, while the programs that produced them continue their work immediately.

Spooling turns a dedicated device into an apparently shared one: ten students in the cybercafé can all "print" at the same time, because in reality they are only writing to the disk queue. The printer then serves the jobs in order. This also explains why a program reports "printing finished" long before the last page physically leaves the printer: the program finished spooling, not printing.



Printer spooling: jobs from several users are queued on disk and printed one after another.

1.7 I/O Scheduling

At any moment, several programs may be competing for the same device: the browser wants to download, the music player wants to read the next song, the antivirus wants to scan. The OS therefore maintains a **request queue** for each shared device and applies an **I/O scheduling** policy to decide which request is served next.

The goals of I/O scheduling are:

- **fairness**: no program should wait forever (avoid starvation);
- **efficiency**: minimise the total time, especially the costly mechanical movements of a disk's read/write head;
- **priority**: urgent system tasks may overtake ordinary ones.

EXEMPLE **First-Come, First-Served scheduling**

Suppose a disk head is at track 50 and four requests arrive in this order for tracks 90, 20, 70, 40. With the simplest policy, First-Come, First-Served (FCFS), the OS serves them exactly in arrival order:

$$50 \rightarrow 90 \rightarrow 20 \rightarrow 70 \rightarrow 40$$

Total head movement: $40+70+50+30 = 190$ tracks. FCFS is perfectly fair but can be slow; cleverer policies (serving nearby tracks first) reduce movement but risk starving far-away requests. The OS must balance the two.

WORKED EXAMPLE 1 **Classifying devices**

For each device, state whether it is (i) input, output or storage; (ii) character or block oriented; (iii) dedicated or shared: a) a keyboard; b) a hard disk; c) a printer.

SOLUTION

a) Keyboard: input device; character device (it delivers a stream of keystrokes, one byte at a time, not addressable); dedicated in practice (it serves the program that currently has the focus). b) Hard disk: storage device (both read and written); block device (data transferred in fixed-size, addressable blocks, allowing random access); shared (the OS interleaves block requests from many programs). c) Printer: output device; character oriented (it receives a stream of bytes describing the page); dedicated (only one job may print at a time — which is precisely why spooling is used to share it).

EXERCISE 1 **At the cybercafé**

Back in the Buea cybercafé: 1) Explain, using the vocabulary of this section, why the 200-page document does not block the other students' computers while it prints. 2) The flash drive became unreadable after a power cut during copying. Which technique seen in this section involves data waiting temporarily in memory before reaching the device, and why does a power cut at that moment endanger the data? 3) Give the layer of the I/O architecture that would contain the code specific to the exact model of the café's printer.

SOLUTION **Exercise 1**

1) Thanks to spooling: the document is first written to a print queue on disk, so the student's program finishes immediately and the printer serves the queued jobs one at a time; other users' jobs simply join the queue. 2) Buffering (and the disk cache): data being copied may still be sitting in a memory buffer, not yet physically written to the flash drive.

A power cut erases the memory buffer, so part of the file never reaches the drive, leaving it incomplete or "corrupted". This is why one should always "safely remove" a USB drive. 3) The device driver layer: it contains the model-specific code translating the OS's general print orders into commands for that particular printer.

KEY POINTS **To remember**

- The I/O subsystem hides hardware differences and offers a uniform interface: open, read, write, close.
- Devices are classified by direction (input/output/storage), by transfer unit (character vs block) and by sharing (dedicated vs shared).
- The controller is hardware; the driver is software that knows one device; together they connect the device to the OS.
- Buffering smooths transfers, caching reuses copies, spooling queues jobs for dedicated devices such as printers.
- The OS schedules competing I/O requests to be both fair and efficient.

2 I/O TRANSFER MECHANISMS

In the previous section we examined how the operating system identifies, configures and abstracts hardware devices. Now we turn to the core question: how does data actually move between a device and main memory? The answer determines whether the CPU spends its cycles waiting, responding to signals, or delegating the work entirely. Three fundamental mechanisms exist — **programmed I/O (polling)**, **interrupt-driven I/O** and **direct memory access (DMA)** — each striking a different balance between simplicity, CPU utilisation and throughput.

2.1 Programmed I/O (Polling)

The simplest way to transfer data is to let the CPU actively check the device's status register until the device signals that it is ready. This is called **programmed I/O** or **polling**.

EXEMPLE **Polling a keyboard**

Imagine a program that reads a character from the keyboard. The keyboard controller has a status register with a `READY` bit. The CPU executes a tight loop:

1. Read the status register.

2. If `READY` = 0, go back to step 1.
3. If `READY` = 1, read the data register and store the character in memory.

While the loop runs, the CPU cannot do any useful work — it is busy-waiting.

Polling is easy to implement and requires no special hardware support beyond a status register. It is acceptable when:

- The device is very fast (e.g., a memory-mapped sensor that responds in nanoseconds).
- The wait time is predictable and short.
- The system has no other tasks to run (bare-metal embedded systems).

However, for slow devices like keyboards, printers or disks, polling wastes enormous amounts of CPU time. The CPU might check the status millions of times before the device becomes ready.

2.2 Interrupt-Driven I/O

To avoid busy-waiting, the device can notify the CPU when it is ready. This notification is called an **interrupt**.

DEFINITION **Interrupt**

An **interrupt** is a hardware signal sent by a device (or by the CPU itself) to the processor, indicating that an event requires immediate attention. The processor suspends its current activity, saves its state, and executes a special routine called an **interrupt service routine (ISR)** or **interrupt handler**.

Interrupt handling steps:

1. The device finishes its operation (e.g., a key is pressed, a disk sector is read) and asserts the interrupt request line (IRQ).
2. The CPU finishes the current instruction, saves the program counter and status register (often on the stack), and disables further interrupts (or masks lower-priority ones).
3. The CPU loads the address of the ISR from the **interrupt vector table** (or interrupt descriptor table) and jumps to it.
4. The ISR runs: it reads the data from the device, stores it in a buffer, updates flags, and possibly wakes up a waiting process.
5. The ISR executes a `return from interrupt` instruction, restoring the saved state and re-enabling interrupts. The interrupted program resumes.

Interrupt-driven I/O eliminates busy-waiting: the CPU can execute other processes while the device works. It is the standard mechanism for character-oriented devices (keyboard, mouse,

serial ports) and for signalling completion of block transfers initiated by DMA.

PROPERTY **Vectored vs. non-vectored interrupts**

In vectored interrupts, the device supplies an interrupt vector (index) that directly identifies the ISR address via the interrupt vector table. In non-vectored interrupts, the CPU must poll devices to find the source. Modern systems use vectored interrupts (e.g., x86 IDT, ARM VIC) for speed.

2.3 Direct Memory Access (DMA)

For high-speed block devices (hard disks, SSDs, network cards, graphics cards), even interrupt-driven I/O imposes too much overhead: the CPU would have to execute an interrupt for every byte or word transferred. **Direct Memory Access (DMA)** allows the device to transfer entire blocks of data directly to/from memory without CPU intervention.

DEFINITION **DMA (Direct Memory Access)**

DMA is a transfer mechanism in which a specialised hardware unit, the **DMA controller**, takes control of the system bus to move data between an I/O device and main memory, while the CPU is free to execute other instructions. The CPU only initiates the transfer (by programming the DMA controller) and is interrupted when the whole block is done.

PROPERTY **DMA improves throughput**

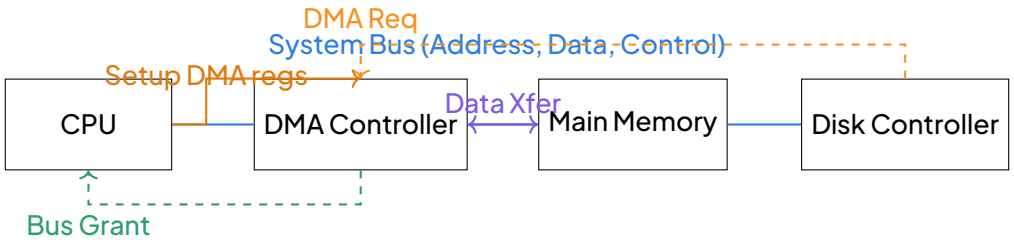
During a DMA transfer, the CPU is not involved in moving each word. It only loses a few bus cycles when the DMA controller requests the bus (cycle stealing) or when the bus is arbitrated. This dramatically increases overall system throughput, especially for large block transfers.

Role of the DMA controller:

The DMA controller is a programmable device that holds:

- The **memory address** (source or destination).
- The **byte/word count** (how many units to transfer).
- The **mode** (read from device / write to device, single/block/demand transfer).
- The **device identifier** (which I/O port).

The CPU sets up these registers, then tells the device to start. The DMA controller and the device then perform the transfer over the bus, using cycle stealing (interleaving bus cycles with the CPU) or burst mode (taking the bus for the whole block).

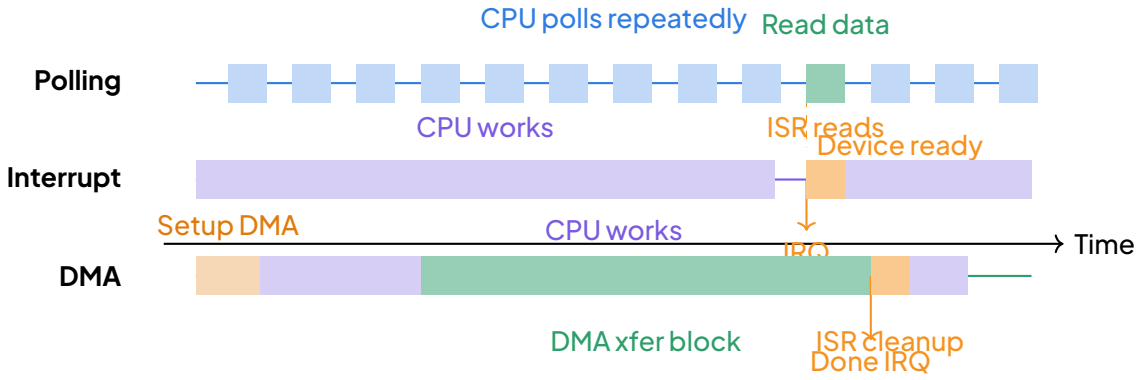


DMA transfer: the CPU programs the DMA controller, then the DMA controller moves data directly between disk and memory over the system bus.

2.4 Comparison of the Three Mechanisms

The table below summarises the key differences. Study it carefully — examination questions often ask you to choose the best mechanism for a given device.

Aspect	Programmed I/O (Polling)	Interrupt-Driven I/O	DMA
CPU involvement	High (busy-wait loop)	Low (only during ISR)	Very low (only setup + completion interrupt)
Hardware complexity	Minimal (status register)	Interrupt controller, vector table	DMA controller, bus arbitration
Transfer speed	Limited by CPU polling rate	Limited by interrupt latency	Very high (bus speed, burst mode)
Typical devices	Simple sensors, LEDs, very fast devices	Keyboard, mouse, serial ports, timer	Disk drives, SSD, network card, GPU, audio
Overhead per byte	Very high	High (interrupt per byte/word)	Negligible (one interrupt per block)



Timeline comparison: Polling wastes CPU cycles; Interrupt allows other work but still per-byte overhead; DMA offloads the whole block.

2.5 Interrupt Priorities and Masking

In a real system, many devices can request interrupts simultaneously. The CPU must decide which to handle first. This is managed by an **interrupt controller** (e.g., the 8259A PIC or the APIC in modern x86).

- **Priority levels:** Each interrupt source is assigned a priority. Higher-priority interrupts can preempt lower-priority ISRs.
- **Masking:** The CPU (or the interrupt controller) can mask (disable) specific interrupt lines or all interrupts below a certain priority. This is used inside critical sections of the kernel.
- **Nested interrupts:** If a higher-priority interrupt arrives while a lower-priority ISR is running, the CPU saves the lower ISR's state and handles the higher one first.

KEY POINTS **Key points on priorities**

- The system timer usually has the highest priority (to maintain the scheduler).
- Device interrupts (disk, network) have medium priority.
- Software interrupts (system calls) often have the lowest priority.
- Masking is essential to protect shared kernel data structures from race conditions.

2.6 Choosing the Right Mechanism: Worked Example

METHOD **Selecting an I/O transfer mechanism**

1. Analyse the device characteristics: Is it character-oriented (byte stream) or block-oriented? What is its data rate? Is the timing predictable?
2. Estimate CPU overhead: How many CPU cycles would polling or per-byte interrupts cost relative to the total CPU capacity?
3. Consider hardware support: Does the system have a DMA controller? Does the device support DMA?
4. Match to mechanism:
   - Polling: Very high-speed, predictable, or simple devices where interrupt overhead exceeds polling cost.
   - Interrupt-driven: Character devices, low-to-medium data rates, unpredictable timing.
   - DMA: High-speed block devices, large transfers, where CPU offload is critical.

5. Validate with OS design: Some OS kernels use hybrid approaches (e.g., polling during high-load network receive — NAPI in Linux).

WORKED EXAMPLE 2 **Mechanism selection for three devices**

Statement: For each of the following devices, recommend the most appropriate I/O transfer mechanism (polling, interrupt-driven, or DMA) and justify your choice.

1. A keyboard connected via USB.
2. A laser printer connected via a parallel port (Centronics).
3. A SATA solid-state drive (SSD) used for the system disk.

SOLUTION

Detailed solution:

1. Keyboard (USB): Interrupt-driven I/O. Reasoning: A keyboard generates characters at human speed (few tens of bytes per second). The timing is completely unpredictable (user presses keys at random). Polling would waste CPU cycles checking for a keypress that rarely occurs. DMA is overkill for single-byte transfers and USB keyboards typically use interrupt endpoints. An interrupt on each keypress (or on a small buffer fill) is ideal.
2. Laser printer (parallel port): Interrupt-driven I/O (or DMA if the printer supports it and the port has a DMA channel). Reasoning: A printer receives a stream of bytes (or compressed page data) at moderate speed (up to a few MB/s). The CPU must send data when the printer's buffer has space. Interrupt-driven I/O allows the CPU to fill the printer's buffer when the printer signals "ready" (via an interrupt). If the printer supports a high-speed mode and the parallel port has a DMA channel, DMA can be used for large raster images, but interrupt-driven is the classic and still common choice.
3. SATA SSD (system disk): DMA (specifically Ultra DMA / AHCI with NCQ). Reasoning: An SSD transfers large blocks (4 KiB pages, often multiple pages per command) at very high speeds (hundreds of MB/s to several GB/s). Using programmed I/O or per-byte interrupts would saturate the CPU and limit throughput. The DMA controller (integrated in the SATA/AHCI host controller) moves data directly between the SSD and memory. The CPU only sets up the command (address, sector count) and receives a single completion interrupt per command (or per batch with NCQ). This is the only mechanism that achieves the device's potential performance.

2.7 Common Pitfalls

WARNING **Polling is not always wrong**

A frequent mistake is to claim that "polling is inefficient and should never be used". In reality:

- Polling avoids the overhead of interrupt handling (context save/restore, vector lookup, cache effects).
- For very high-speed devices (e.g., 100 Gbps network cards, GPU command submission), the interrupt rate would be so high that the CPU spends more time handling interrupts than processing data. Modern high-performance drivers often use polled mode (or a hybrid: interrupt for the first packet, then poll for a batch) — this is the basis of technologies like DPDK and NAPI.
- In real-time systems with deterministic deadlines, polling can guarantee a maximum response time, whereas interrupts introduce jitter.

Exam tip: If asked to justify polling, mention low latency, determinism, or avoiding interrupt overhead at very high data rates.

DID YOU KNOW? **Did you know?**

The first DMA controller in the IBM PC was the Intel 8237, a 4-channel chip that could only address 16-bit addresses (64 KiB) and required an external page register to reach the full 1 MiB of the 8088. Modern systems integrate dozens of DMA channels into the chipset (e.g., Intel's I/OAT, AMD's IOMMU) and support scatter-gather lists, allowing a single DMA operation to transfer data to/from non-contiguous memory buffers — essential for zero-copy networking and virtualization.

EXERCISE 2 **Practice questions**

1. Explain why a mouse driver typically uses interrupt-driven I/O rather than polling or DMA.
2. A sound card plays audio by sending samples to a DAC at 48 kHz (stereo, 16-bit). The CPU must supply a new sample pair every 20.8 μs. Which transfer mechanism would you choose and why?
3. Describe the sequence of events when a disk controller finishes a DMA read operation and signals completion.
4. In a system with two interrupting devices (network card priority 3, timer priority 1), the CPU is executing the network ISR when the timer interrupt arrives. What happens? (Assume lower number = higher priority.)

5. A student writes: "DMA is always faster than interrupt-driven I/O because the CPU does not touch the data." Identify the flaw in this statement and give a counter-example.

SOLUTION **Exercise 1**

1. A mouse sends small packets (3–4 bytes) at irregular intervals (up to 1000 Hz). Polling would waste CPU cycles; DMA is unnecessary for such tiny transfers. Interrupt-driven I/O lets the CPU sleep until a packet arrives, then quickly read it.
2. Audio playback requires a steady, low-latency stream. The period (20.8 μs) is short but predictable. Interrupt-driven I/O with a double buffer is common: the sound card interrupts when half the buffer is empty, the ISR fills that half. DMA is also used (the sound card has its own DMA engine) — the CPU sets up a circular buffer and the sound card's DMA fetches samples autonomously. Both are acceptable; pure polling would be risky due to scheduling jitter.
3. 1) Disk controller asserts IRQ. 2) CPU completes current instruction, saves context, jumps to disk ISR. 3) ISR reads DMA status register to verify success, clears the interrupt at the controller, updates the request queue, wakes the waiting process. 4) ISR returns, scheduler may switch to the awakened process.
4. The timer has higher priority (1 < 3). The CPU will preempt the network ISR: save its state, run the timer ISR, then return to the network ISR. This ensures the system clock stays accurate.
5. The flaw: DMA still requires CPU involvement for setup and completion interrupt. For very small transfers (e.g., 1 byte), the setup overhead (programming registers, cache flushes, bus arbitration) exceeds the cost of a simple interrupt-driven read. Counter-example: reading a single byte from a UART — interrupt-driven is faster and simpler.

3 FILE SYSTEMS AND FILE MANAGEMENT

In the previous section we studied how data physically travels between the processor and the peripherals, using programmed I/O, interrupt-driven I/O and DMA. But once bytes can be moved to and from a disk, a new question arises: how are those bytes organised so that users can find, store and protect their information? A raw disk is nothing more than a huge sequence of numbered blocks. Without any organisation, finding "my biology notes" would mean remembering something like "blocks 4521 to 4532 of disk 0". Nobody wants that. The part of the operating system that turns this ocean of blocks into a clean world of **files** and **folders** is the **file system**. It is the librarian of the computer: it catalogues every document, knows exactly where each one is shelved, and keeps a register of the free shelves.

3.1 The File Concept

DEFINITION **File**

A **file** is a named collection of related information stored on a secondary storage device. From the user's point of view, a file is the smallest unit of logical storage: data cannot be kept on a disk unless it is placed inside a file.

A file is much more than its content. Every file carries **attributes** (also called metadata), which the file system stores separately from the data itself:

- **Name**: the human-readable identifier, e.g. `report.docx`.
- **Type**: the kind of content, often indicated by the extension (`.txt`, `.jpg`, `.exe`, `.mp3`) or by internal "magic bytes".
- **Size**: the number of bytes currently used.
- **Location**: pointers to the physical blocks holding the data (hidden from the user).
- **Dates**: creation, last modification, last access.
- **Permissions**: who may read, write or execute the file (owner, group, others).

Common file types include text files, executable programs, images, audio/video files, spreadsheets, databases and special files (directories themselves, and device files such as `/dev/sda` under Linux).

WARNING **Name is not location**

A very common mistake is to believe that the **file name** is the file or tells you where the file physically sits on the disk. It does not. The name is only an entry in a directory; the file system uses it to look up the physical block numbers, which may be scattered anywhere on the disk and may change when the file is edited, moved between folders or defragmented. Renaming a file changes nothing about where its bytes are stored.

3.2 File Operations

Programs manipulate files through system calls provided by the operating system. The most fundamental operations are:

- `create`: make a new, empty file and its directory entry.
- `open`: check permissions, load the file's attributes into memory and return a **file descriptor** (or handle) — a small integer that identifies the open file in all later operations.
- `read` / `write`: transfer data between the file and memory, advancing a current-position pointer.
- `seek` (or `lseek`): move the current-position pointer to a given offset (random access).

- `close`: release the descriptor and flush buffers to disk.
- `delete` (or `unlink`): remove the directory entry and free the blocks.
- `rename`: change the name in the directory entry without touching the data blocks.
- `truncate`: reduce the file size to a given length, freeing excess blocks.
- `stat` / `fstat`: retrieve the file's attributes (metadata).

EXEMPLE **Typical program logic using a file descriptor**

- `fd ← open("marks.csv", READ_ONLY)`
- `data ← read(fd, 1024)`
- `process(data)`
- `close(fd)`

The descriptor `fd` is used exactly like a receipt: once the file is open, the program never mentions the name again — it simply presents the descriptor for each operation.

WARNING **Forgetting to close files**

Two frequent errors: (1) forgetting `close`, which may leave buffered data unwritten and exhaust the limited table of descriptors; (2) using a file name after opening instead of the descriptor, or using a descriptor after closing it — both cause runtime errors.

3.3 Functions of a File System

DEFINITION **File system**

A **file system** is the component of the operating system that organises, stores, retrieves, names, shares and protects files on a storage device. Its main functions are:

1. Naming: mapping user-friendly names to files.
2. Storage allocation: deciding which disk blocks hold each file's data.
3. Directory management: maintaining the structure that groups and locates files.
4. Free-space tracking: recording which blocks are available for new data.
5. Protection and reliability: enforcing permissions and surviving failures.

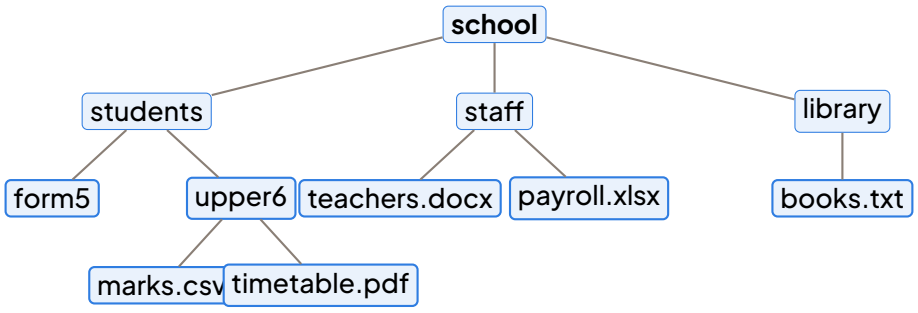
3.4 Directory Structures and Paths

DEFINITION **Directory and path**

A **directory** (folder) is a special file that contains a list of file names together with references to their attributes and locations. A **path** is the sequence of directories that must be traversed to reach a file. An **absolute path** starts from the root directory (e.g. `/home/ayi/maths/notes.txt` or `C:\Users\Ayi\notes.txt`); a **relative path** starts from the current working directory (e.g. `maths/notes.txt` if we are already in `/home/ayi`).

Several directory structures have been used historically:

- **Single-level directory**: one directory for all users and all files. Simple, but two files cannot share a name — impossible as soon as several users exist.
- **Two-level directory**: one master directory pointing to one directory per user. Solves name clashes between users, but a user still cannot organise his own files.
- **Tree-structured directory**: directories may contain other directories, forming a tree rooted at `/` (or `C:\`). This is the structure of all modern systems; it allows natural grouping and efficient searching.
- **Acyclic-graph directory**: like a tree, but a file or directory may be shared (linked) so that it appears in several directories. Powerful, but deletion and bookkeeping become delicate (when is the file really no longer needed?).



A tree-structured directory for a school. The absolute path of the marks file is `/school/students/upper6/marks.csv`.

EXEMPLE **Absolute versus relative path**

Suppose the current working directory is `/school/students`. To open the marks file we may write the absolute path `/school/students/upper6/marks.csv`, or the relative path `upper6/marks.csv`. The notation `..` means "the parent directory": from `upper6`, the payroll file can be reached relatively as `../../staff/payroll.xlsx`.

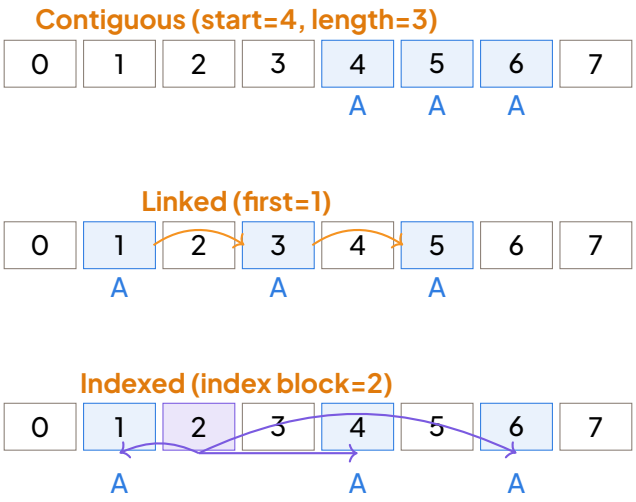
3.5 File Allocation Methods

The disk is divided into fixed-size **blocks** (also called clusters in some systems, typically 4 KB). The central design problem is: which blocks do we give to each file? Three classic methods exist.

Contiguous allocation. Each file occupies a single run of consecutive blocks. The directory only stores the starting block and the length (in blocks).

Linked allocation. Each file is a linked list of blocks scattered anywhere; every block ends with a pointer to the next block. The directory stores only the first (and usually last) block number.

Indexed allocation. Each file has an index block containing the addresses of all its data blocks. The directory stores the index block number; random access to block *i* is immediate via the *i*-th entry of the index.



The same 3-block file `A` stored with contiguous, linked and indexed allocation.

PROPERTY **Contiguous allocation: fast but fragile**

With contiguous allocation, both sequential and direct access are very fast (block *i* is simply at address start + *i*), and the disk head moves very little. However, as files are created and deleted, the disk ends up with many small holes that are individually too small for new files: this is **external fragmentation**. In addition, the file's maximum size must be known in advance, and growing a file may require copying it elsewhere. (Statement examinable; no formal proof required.)

Criterion	Contiguous	Linked	Indexed
Sequential access	Excellent	Good (follow pointers)	Good
Direct (random) access	Excellent	Poor (must walk the chain)	Excellent
External fragmentation	Severe	None	None
Space overhead	None	One pointer per block	One index block per file
File growth	Difficult	Easy	Easy (within index capacity)
Reliability	Good	A lost pointer loses the rest of the file	Index is a single point of failure

METHOD **Choosing an allocation method**

1. Identify the dominant access pattern: mostly sequential (logs, videos) or random (databases)?
2. Estimate whether files grow after creation and how often they are deleted (fragmentation risk).
3. If files are static and read sequentially or randomly ⇒ contiguous.
4. If files grow unpredictably and are read sequentially ⇒ linked.
5. If fast random access to large, growing files is needed ⇒ indexed (the choice of most modern systems).

3.6 Free-Space Management

The file system must always know which blocks are free. Two classical techniques are used:

- **Bit map (bit vector)**: one bit per block — 1 means free, 0 means occupied (or the reverse). A disk of 2^{20} blocks needs only 2^{20} bits = 128 KB, which fits in main memory, so finding free blocks is fast. Contiguous free space, however, requires scanning for a run of 1s.
- **Linked free list**: all free blocks are chained together, each free block holding the address of the next free block. No extra space is needed, but traversing the list is slow and finding contiguous space is difficult.

WORKED EXAMPLE 3 **Size of a bit map**

A disk has 16 GB of capacity formatted in blocks of 4 KB. How large is the bit map used to track free blocks?

SOLUTION

Number of blocks: $\frac{16 \times 2^{30}}{4 \times 2^{10}} = \frac{2^{34}}{2^{12}} = 2^{22}$ blocks.
The bit map needs one bit per block: 2^{22} bits = $2^{22}/2^{3} = 2^{19}$ bytes = 512 KB.
This is tiny compared with 16 GB, which is why bit maps are kept in memory while the disk is in use.

3.7 Common File Systems and Journaling

- **FAT32** (File Allocation Table, 32-bit): an old, very simple linked-allocation system using a table of block pointers. Universally readable (USB flash drives, memory cards) but limited to files of at most 4 GB and offering no permissions or journaling.
- **NTFS** (New Technology File System): the standard Windows file system. It supports very large files and disks, access-control lists, compression, encryption and journaling.
- **ext4** (Fourth Extended File System): the standard Linux file system. It uses extents (runs of contiguous blocks) to reduce fragmentation, supports huge volumes and provides journaling.

DEFINITION **Journaling**

A **journaling file system** keeps a small log (the journal) in which every intended modification is written before it is applied to the main file system structures. After a power failure or crash, the system simply replays or discards the incomplete operations recorded in the journal, instead of checking the whole disk. The file system is therefore restored to a consistent state within seconds.

DID YOU KNOW? **Why your phone survives a flat battery**

Without journaling, unplugging a computer in the middle of a save could leave a directory pointing to blocks that were never written — a corrupted file system requiring a long repair scan. Thanks to journaling in ext4 and NTFS, modern laptops and Android phones (which use ext4-family systems) usually restart cleanly even after the battery dies during an update.

3.8 Protection and Access Control

Modern file systems associate **access control lists** (ACLs) or traditional **permission bits** with each file and directory. The classic Unix model distinguishes three classes of users:

- **Owner** (user who created the file)

- **Group** (a set of users sharing access rights)
- **Others** (everyone else)

For each class, three permissions may be granted or denied:

- **Read (r)**: view file contents or list directory entries.
- **Write (w)**: modify file contents or create/delete files in a directory.
- **Execute (x)**: run the file as a program or traverse (enter) a directory.

EXEMPLE **Permission notation**

The string `rwxr-xr--` means:

- Owner: read, write, execute
- Group: read, execute
- Others: read only

In octal, this is `754` (7 = 4 + 2 + 1, 5 = 4 + 0 + 1, 4 = 4 + 0 + 0).

WARNING **Directory execute permission**

A common trap: to enter a directory (cd into it) or access files inside it, a user needs execute (x) permission on that directory, not just read permission. Read permission on a directory only allows listing its entry names.

KEY POINTS **Key points — File Systems and File Management**

- A file = data + attributes (name, type, size, dates, permissions); the name is not the physical location.
- Directories organise files; modern systems use tree structures with absolute and relative paths.
- File operations: create, open, read, write, seek, close, delete, rename, truncate, stat.
- Contiguous allocation is fast but causes external fragmentation; linked allocation removes fragmentation but is poor for random access; indexed allocation combines flexibility and fast direct access at the cost of an index block.
- Free space is tracked with bit maps or linked free lists.
- FAT32, NTFS and ext4 are the common file systems; journaling guarantees fast recovery after a crash.

- Protection uses permission bits (r/w/x) for owner/group/others or ACLs; directory execute permission is required to traverse it.

EXERCISE 3 **Allocation and paths**

A file `F` of 10 KB is stored on a disk with 2 KB blocks.

1. How many blocks does `F` occupy?
2. Give the directory information needed under contiguous, linked and indexed allocation.
3. From the directory `/school/staff` in the figure above, write the relative path to `marks.csv`.

SOLUTION **Exercise 1**

1. 10 KB / 2 KB = 5 blocks.
2. Contiguous: starting block and length (5). Linked: number of the first block (and ideally the last). Indexed: number of the index block, which lists the 5 data-block addresses.
3. `../students/upper6/marks.csv`.

4 MASS STORAGE AND FILE ACCESS METHODS

Having understood how files are organised logically (file systems) and physically (allocation methods), we now turn to the hardware that holds the files — mass storage devices — and the access methods that determine how programs read and write file data.

4.1 Mass Storage Devices

DEFINITION **Mass storage**

Mass storage (or secondary storage) refers to non-volatile media that retain data without power and provide much larger capacity than main memory (RAM), at a lower cost per bit but with higher access latency.

The main technologies encountered in the GCE syllabus are:

1. **Magnetic Hard Disk Drives (HDD)**:
   - Rotating platters coated with magnetic material; read/write heads on an actuator arm.
   - Data organised in **tracks** (concentric circles), **sectors** (pie-slices), and **cylinders** (tracks at same radius on all platters).
   - Typical capacity: 500 GB – 20 TB. Latency: 3–10 ms (seek + rotational). Throughput: 100–250 MB/s.
   - Vulnerable to shock; mechanical wear.
2. **Solid State Drives (SSD)**:
   - NAND flash memory, no moving parts. Data stored in **pages** (e.g. 4 KB) grouped into **blocks** (e.g. 256 KB).
   - **Wear levelling** and **TRIM** commands extend lifetime.
   - Typical capacity: 128 GB – 8 TB. Latency: 0.05–0.1 ms. Throughput: 500 MB/s – 7 GB/s (NVMe).
   - More expensive per GB than HDD but faster, silent, shock-resistant.
3. **Optical Discs** (CD, DVD, Blu-ray):
   - Laser reads/writes pits/lands on reflective spiral track.
   - CD: 700 MB, DVD: 4.7–8.5 GB, Blu-ray: 25–100 GB.
   - Mostly read-only or write-once (R) / rewritable (RW). Used for distribution, backup, archival.
4. **Magnetic Tape**:
   - Sequential access only. Very high capacity per cartridge (LTO-9: 18 TB native, 45 TB compressed).
   - Extremely low cost per GB, long archival life (30+ years).
   - Used for enterprise backup and cold storage; access time: tens of seconds.
5. **USB Flash Drives / Memory Cards**:
   - NAND flash in portable form factors (USB-A, USB-C, SD, microSD).
   - Convenient for data transfer between devices; not for primary storage.

Characteristic	HDD	SSD (SATA/NVMe)	Optical	Tape
Access type	Direct (random)	Direct (random)	Direct (random)	Sequential
Typical latency	3–10 ms	0.05–0.1 ms	100–200 ms	10–100 s
Throughput	100–250 MB/s	500 MB/s – 7 GB/s	10–50 MB/s	300–1000 MB/s
Cost per GB	Low	Medium–High	Low (media)	Very low
Durability	Mechanical wear	Write endurance limits	Scratch sensitive	Excellent (archival)
Typical use	Bulk storage, servers	OS, apps, databases	Distribution, backup	Long-term backup

4.2 File Access Methods

The **file access method** defines how a program reads or writes the logical records of a file. The choice depends on the application and the storage device.

DEFINITION **Sequential access**

Sequential access processes records one after another, from the beginning to the end. The file pointer advances automatically after each read/write. To reach record n, the first $n - 1$ records must be read (or skipped).

- **Use cases**: log files, print spooling, video/audio streaming, batch processing, tape backups.
- **Devices**: optimal on tape; works on any device.
- **Operations**: `read_next`, `write_next`, `rewind`, `forward/backward n records`.

DEFINITION **Direct (random) access**

Direct access (or **random access**) allows jumping immediately to any record by its address (record number or byte offset). The file is viewed as a numbered array of blocks/records.

- **Use cases**: databases, file systems, compilers (symbol tables), any application needing fast lookup.
- **Devices**: requires direct-access storage (disk, SSD, optical). Impossible on tape.
- **Operations**: `read(n)`, `write(n)`, `seek(position)`.
- **Implementation**: the file system translates logical record number n into physical block address using the allocation method (contiguous: start+n; indexed: index block entry n; linked: must traverse — hence linked is poor for direct access).

DEFINITION **Indexed Sequential Access Method (ISAM)**

ISAM combines sequential and direct access. The file is stored sequentially on disk, but a separate index (often a B-tree or multi-level index) maps key values to record locations. This allows both fast key-based lookup and efficient sequential processing in key order.

- **Structure**: data area (sequential by key) + index area (key → pointer).
- **Operations**: `read_by_key(key)`, `read_next`, `insert`, `delete` (with overflow handling).
- **Use cases**: traditional database files (VSAM, COBOL ISAM), library catalogues, inventory systems.
- **Modern evolution**: B^+-trees (used in NTFS, ext4, databases) generalise ISAM with balanced tree indexes that handle dynamic insertion/deletion gracefully.

Criterion	Sequential	Direct (Random)	Indexed Sequential (ISAM)
Access pattern	Linear, start to end	Arbitrary jumps	Key-based lookup + sequential
Storage requirement	Any (incl. tape)	Disk, SSD, optical	Disk, SSD
Search speed	$O(n)$	$O(1)$ (with index)	$O(\log n)$ (tree index)
Insert/Delete	Append only (or rewrite)	In-place (if space)	Complex (overflow blocks)
Typical file organisation	Flat, unordered	Fixed-length records	Sorted by key + index
Example	Log file, CSV export	Database table, RAM file	COBOL/VSAM dataset

METHOD **Choosing a file access method**

1. Analyse the application: Does it process all records in order (report, backup) → sequential? Does it need to fetch/update individual records by ID/key (database, dictionary) → direct or ISAM?
2. Check the storage medium: Tape forces sequential. Disk/SSD allows all three.
3. Consider record structure: Fixed-length records simplify direct access (record n at offset $n \times$ len). Variable-length records usually need an index.
4. Estimate concurrency: ISAM/B-tree indexes handle concurrent access better than simple sequential files.

4.3 Logical vs Physical Records and Blocking

DEFINITION **Logical record, physical block, blocking factor**

A **logical record** is the unit of data meaningful to the application (e.g. one student record). A **physical block** (or sector) is the unit of transfer between disk and memory (e.g. 4 KB). **Blocking** groups several logical records into one physical block. The **blocking factor** b is the number of logical records per block.

EXEMPLE **Blocking calculation**

A student record is 200 bytes. Disk block size = 4 KB = 4096 bytes. Blocking factor $b = \lfloor 4096/200 \rfloor = 20$ records per block. Wasted space per block = $4096 - 20 \times 200 = 96$ bytes (internal fragmentation). Reading one block transfers 20 records at once, reducing I/O calls by a factor of 20.

WARNING **Blocking trade-off**

Larger blocking factor ⇒ fewer I/O operations, better throughput, but more memory for buffers and higher latency to retrieve the first record of a block. The OS typically chooses a block size (4 KB, 8 KB, 64 KB) that balances these factors.

4.4 Buffering, Caching and Read-Ahead

Because disk I/O is orders of magnitude slower than CPU/RAM, the OS uses several techniques to hide latency:

- **Buffering**: a region in kernel memory holds data during transfer between device and user process. Single buffering, double buffering (overlap I/O of block $n + 1$ with processing of block n), circular buffering (multiple blocks).
- **Caching (disk cache / page cache)**: recently used blocks are kept in RAM. Subsequent reads hit in cache (nanoseconds) instead of going to disk (milliseconds). Write-back caching delays writing dirty blocks to disk (risk on power loss).
- **Read-ahead (prefetching)**: when sequential access is detected, the OS speculatively reads the next few blocks into cache before the application asks for them.

DID YOU KNOW? **Why `fsync` matters**

When you save a document, the data often goes only to the OS page cache in RAM. The `fsync()` system call forces the OS to flush that data to the physical disk. Databases call

`fsync` after each transaction to guarantee durability (the D in ACID). Without it, a power cut loses the last seconds of work.

KEY POINTS **Key points — Mass Storage and File Access Methods**

- Main mass storage: HDD (magnetic, mechanical, cheap, high capacity), SSD (flash, fast, no moving parts), Optical (removable, archival), Tape (sequential, very high capacity, cold storage).
- Sequential access: process records in order; only method possible on tape.
- Direct (random) access: jump to any record by address; requires disk/SSD; essential for databases.
- Indexed Sequential (ISAM): sequential storage + tree index for key-based lookup; precursor to modern B^+-tree file organisations.
- Blocking groups logical records into physical blocks; blocking factor $b = \lfloor \text{block size} / \text{record size} \rfloor$.
- Buffering, caching and read-ahead hide disk latency; `fsync` ensures durability.

EXERCISE 4 **Mass storage and access methods**

1. A magnetic tape cartridge holds 18 TB native. A company produces 2 TB of backup data per day. How many cartridges are needed for a 7-day full backup cycle if each cartridge is used once per cycle?
2. A file contains 50,000 fixed-length records of 120 bytes each. The disk block size is 4 KB.
   (a) What is the blocking factor?
   (b) How many blocks does the file occupy?
   (c) If the file is accessed sequentially, how many I/O operations are needed to read the whole file?
   (d) If the file is accessed directly (random) and the index is in memory, how many I/O operations to read one record?
3. Which access method (sequential, direct, ISAM) is most suitable for:
   (a) A payroll program that prints payslips for all employees in employee-number order?

(b) An airline reservation system that checks seat availability for a given flight number?

(c) A library catalogue that supports both "find book by ISBN" and "list all books by author X"?

SOLUTION **Exercise 2**

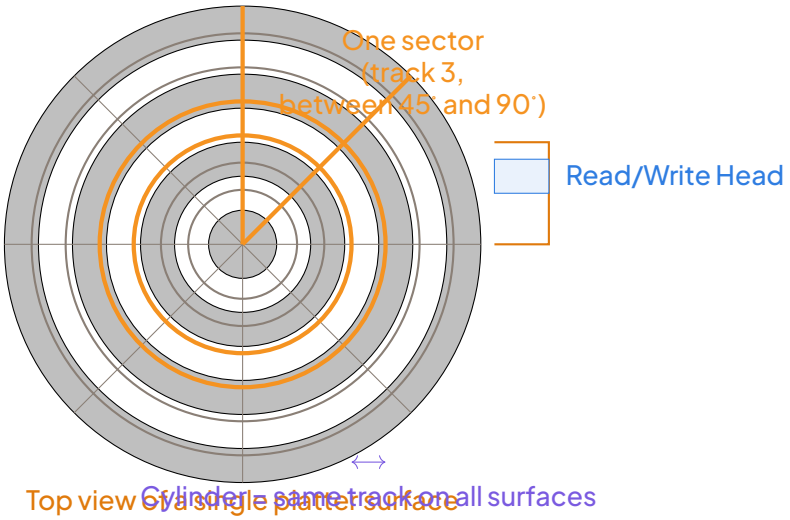
1. 7 days × 2 TB/day = 14 TB. One cartridge holds 18 TB ⇒ 1 cartridge suffices for the 7-day cycle (with 4 TB spare).
2. (a) Blocking factor $b = \lfloor 4096/120 \rfloor = 34$ records/block.
   (b) Blocks needed = $\lceil 50\,000/34 \rceil = 1471$ blocks.
   (c) Sequential read: 1471 I/O operations (one per block).
   (d) Direct read of one record: 1 I/O operation (read the block containing that record), assuming index in memory.
3. (a) Sequential (process all records in order).
   (b) Direct (random access by flight number).
   (c) ISAM (or B^+-tree): key-based lookup by ISBN + sequential scan for author.

5 MASS STORAGE AND FILE ACCESS METHODS

In the previous section we saw how the operating system organises files logically through directories, metadata and allocation tables. That logical view, however, must eventually be mapped onto physical devices that can retain data without power. This section explores the hardware technologies behind mass storage, the performance characteristics that drive operating-system design, and the three fundamental ways programs access stored data.

5.1 Structure of Magnetic Disks

Although solid-state drives are now common, the traditional magnetic hard disk drive (HDD) remains the reference model for understanding mass-storage geometry and performance. A clear mental picture of its moving parts explains why seek time and rotational latency dominate I/O scheduling.



Geometry of a magnetic disk surface: tracks (concentric circles), sectors (pie slices), and the read/write head. A cylinder is the set of tracks at the same radius on all platters.

DEFINITION **Magnetic Disk Geometry**

A magnetic hard disk consists of one or more rigid **platters** coated with magnetic material. Each platter has two **surfaces** (top and bottom). Data is recorded on concentric circles called **tracks**. Each track is divided into fixed-size **sectors** (typically 512 bytes or 4 KiB). The set of tracks at the same radius on all surfaces forms a **cylinder**. The **read/write head** floats on an air bearing above each surface; all heads move together as a single assembly (the **actuator arm**).

PROPERTY **Disk Addressing – CHS and LBA**

Historically, a sector was addressed by three numbers: **Cylinder** (which cylinder), **Head** (which surface/head), and **Sector** (which sector on that track) – the **CHS** (Cylinder-Head-Sector) scheme. Modern disks use **Logical Block Addressing (LBA)**: sectors are numbered sequentially from 0 to $N - 1$, hiding the physical geometry from the OS. The disk controller translates LBA to physical location internally.

5.2 Disk Performance: Seek Time, Rotational Latency, Transfer Time

Because the head must physically move and the platter must spin, every I/O request incurs three delays.

DEFINITION **Seek Time**

Seek time is the time required to move the actuator arm so that the head is positioned over the target track. It depends on the distance (number of cylinders) traversed. Manufacturers quote **average seek time** (typically 3–10 ms for enterprise HDDs) and **track-to-track seek time** (0.5–1 ms).

DEFINITION **Rotational Latency**

Once the head is on the correct track, it must wait for the target sector to rotate under it. **Rotational latency** is this waiting time. On average it is half a revolution:

$$\text{Average rotational latency} = \frac{1}{2} \times \frac{60}{\text{RPM}} \text{ seconds} = \frac{30}{\text{RPM}} \text{ seconds}.$$

For a 7200 RPM disk: $\frac{30}{7200} = 4.17$ ms.

DEFINITION **Transfer Time**

Transfer time is the time to actually read or write the data once the sector is under the head. For a single sector:

$$\text{Transfer time} = \frac{\text{Sector size}}{\text{Transfer rate}}.$$

Transfer rate depends on rotational speed and recording density (typically 100–250 MB/s for modern HDDs). Note: storage transfer rates use decimal megabytes (1 MB/s = 10^6 bytes/s).

KEY POINTS **Average Access Time Formula**

The **average access time** to read or write a randomly located block is:

$$T_{\text{access}} = T_{\text{seek,avg}} + T_{\text{rot,avg}} + T_{\text{transfer}}.$$

For sequential access of consecutive sectors on the same track, seek time and rotational latency are incurred only once; subsequent sectors cost only transfer time.

METHOD **Computing Time to Read a File from Disk**

Given: average seek time T_s, RPM R, sector size S, transfer rate R_{rate} (in bytes/s), and file

size F (in bytes). Assume the file is stored contiguously on one track.

1. Compute average rotational latency: $T_{rot} = \frac{30}{R}$ seconds.
2. Transfer time for the whole file: $T_{xfer} = \frac{F}{R_{rate}}$ (ensure consistent units: bytes and bytes/s).
3. Total time $\approx T_s + T_{rot} + T_{xfer}$.

If the file spans multiple tracks, add one seek and one rotational latency per track switch.

WORKED EXAMPLE 4 **Access Time Calculation**

A 7200 RPM hard disk has an average seek time of 8 ms, a sustained transfer rate of 150 MB/s, and 4 KiB sectors. Estimate the time to read a 200 KiB file stored contiguously on a single track.

SOLUTION

Solution:

- $T_s = 8$ ms.
- Average rotational latency $T_{rot} = \frac{30}{7200} = 4.167$ ms.
- File size $F = 200$ KiB $= 200 \times 1024 = 204\,800$ bytes.
- Transfer rate $R_{rate} = 150$ MB/s $= 150 \times 10^6$ B/s (decimal megabytes).
- Transfer time $T_{xfer} = \frac{204\,800}{150 \times 10^6} = 1.365$ ms.
- Total $\approx 8 + 4.167 + 1.365 = 13.53$ ms.

Answer: Approximately 13.5 ms.

5.3 Solid-State Drives (SSD) vs Hard Disks

SSDs store data in flash memory (NAND) with no moving parts. This fundamentally changes the performance profile.

Characteristic	Magnetic HDD	SSD (NAND Flash)
Moving parts	Yes (spinning platters, actuator)	None
Random access time	5–15 ms (seek + rotation)	0.05–0.15 ms (electronic)
Sequential throughput	100–250 MB/s (SATA)	500–7000 MB/s (NVMe PCIe)
Cost per GB	Lower (≈ $0.02–0.03/GB)	Higher (≈ $0.08–0.15/GB, indicative)
Lifespan (writes)	Limited by mechanical wear	Limited program/erase cycles (TBW rating)
Fragmentation impact	Severe (extra seeks)	Negligible (no seek penalty)
Noise / Power	Audible, higher power	Silent, lower power

WARNING **Common Mistake: SSDs Do Not Have Physical Tracks and Sectors**

Students often describe SSDs using HDD terminology (tracks, sectors, cylinders, seek time). This is incorrect. SSDs present a logical block interface (LBA) to the OS, but internally they use **pages** (e.g. 4–16 KiB) grouped into **blocks** (e.g. 256–512 pages). Writing requires erasing a whole block first. The controller runs a **Flash Translation Layer (FTL)** that maps LBAs to physical pages, performs wear levelling, and manages garbage collection. There is no mechanical seek or rotational latency.

5.4 Other Mass Storage Technologies

- **Optical Discs (CD, DVD, Blu-ray):** Laser reads pits/lands on reflective spiral track. Cheap, removable, good for distribution; slow random access, limited rewrite cycles (CD-RW, DVD-RW). Capacities: 700 MB (CD), 4.7 GB (DVD), 25–100 GB (Blu-ray).
- **USB Flash Drives / Memory Cards (SD, microSD):** NAND flash in portable form factors. No moving parts, convenient for transfer between devices. Speeds vary widely (USB 2.0 to USB 4 / UHS-I/II/III).
- **Magnetic Tape:** Sequential-access medium, very high capacity per cartridge (LTO-9: 18 TB native, 45 TB compressed), very low cost per GB, long archival life (30+ years). Used for backup and cold storage. Access time is minutes (wind/rewind), so unsuitable for random access.

5.5 Disk Scheduling Basics (FCFS and SSTF)

When multiple I/O requests queue for the same disk, the order in which they are serviced dramatically affects total head movement and thus throughput. The OS **I/O scheduler** chooses the order. Two classic algorithms are:

DEFINITION **First-Come, First-Served (FCFS)**

Requests are serviced in arrival order. Simple, fair, no starvation. But head movement can be excessive ("elevator" thrashing).

DEFINITION **Shortest Seek Time First (SSTF)**

The request whose track is closest to the current head position is chosen next. Minimises seek time, improves throughput. Risk: **starvation** of requests far from the current head position if a steady stream of nearby requests arrives.

EXEMPLE **FCFS vs SSTF Trace**

Disk with tracks 0–199. Head starts at 50. Request queue: 82, 170, 43, 140, 24, 16, 190.

- FCFS order: 50 → 82 → 170 → 43 → 140 → 24 → 16 → 190. Total head movement = |82 − 50| + |170 − 82| + |43 − 170| + |140 − 43| + |24 − 140| + |16 − 24| + |190 − 16| = 32 + 88 + 127 + 97 + 116 + 8 + 174 = 642 cylinders.
- SSTF order: 50 → 43 (7) → 24 (19) → 16 (8) → 82 (66) → 140 (58) → 170 (30) → 190 (20). Total = 7 + 19 + 8 + 66 + 58 + 30 + 20 = 208 cylinders.

SSTF reduces movement by 68% here, but if new requests keep arriving near 50, the outer tracks may wait indefinitely.

5.6 File Access Methods

The OS provides abstract ways for programs to read/write files. The choice of access method matches the application's access pattern.

DEFINITION **Sequential Access**

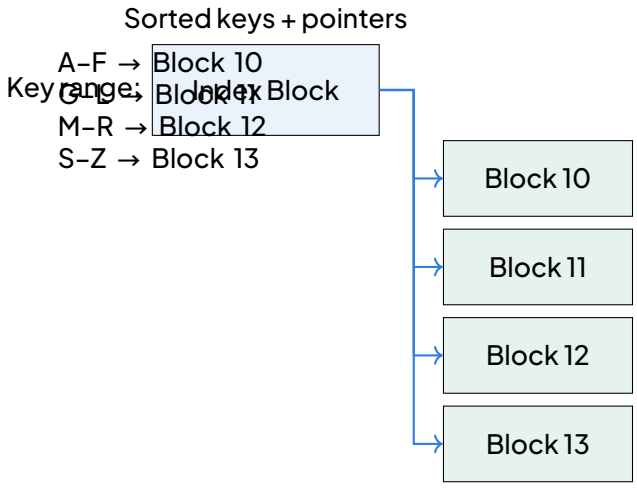
Data is processed in order, one record after another, from beginning to end. The file pointer advances automatically after each read/write. No direct jumping. Analogy: cassette tape.

DEFINITION **Direct (Random) Access**

The file is viewed as a numbered sequence of blocks/records. The program can specify any block number to read/write next. Analogy: CD track selection or array indexing.

DEFINITION **Indexed Sequential Access**

A hybrid: an **index** (like a book's table of contents) maps key values to block addresses. The index itself may be multi-level (e.g. ISAM – Indexed Sequential Access Method). Allows fast key-based lookup and sequential processing in key order.



Indexed sequential access: an index block points to data blocks covering key ranges. Lookup = binary search in index + one direct block read.

PROPERTY **Matching Access Methods to Applications**

- Sequential access suits **batch processing** where the whole file is read/written in order (e.g. payroll run on magnetic tape, printing all student transcripts, generating monthly bank statements).
- Direct access suits **interactive/on-line** applications where a single record must be fetched quickly (e.g. ATM balance inquiry, airline seat reservation, looking up a student by ID).
- Indexed sequential access suits applications needing both keyed lookup and ordered reports (e.g. student records indexed by registration number, but also printed in alphabetical order; inventory system searched by part number but listed by category).

DID YOU KNOW? **Cameroon Context: Mobile Money and Indexed Access**

Mobile money platforms (MTN MoMo, Orange Money) in Cameroon handle millions of accounts. When you dial *150# and check your balance, the system performs a direct access

by your phone number (primary key). When the regulator requests a monthly report of all transactions sorted by date, the system uses an index on the transaction date to avoid a full sequential scan of the huge transaction log.

5.7 Summary and Examination Focus

KEY POINTS **Key Points for GCE Revision**

- Magnetic disk geometry: platter, surface, track, sector, cylinder. Addressing: CHS (legacy) vs LBA (modern).
- Access time = seek time + rotational latency + transfer time. Know the formula and units (ms, RPM, MB/s).
- SSD: no moving parts, electronic access, flash translation layer, wear levelling. Do not describe SSDs with tracks/sectors.
- Other media: optical (distribution), flash drives (portability), tape (backup/archive).
- Disk scheduling: FCFS (fair, simple) vs SSTF (lower seek, starvation risk). Be able to compute total head movement for a given queue.
- File access methods: sequential (batch), direct/random (interactive), indexed sequential (keyed + ordered). Match each to typical applications.

EXERCISE 5 **Practice Questions**

1. A 10 000 RPM disk has average seek time 4 ms. Sectors are 4 KiB, transfer rate 200 MB/s. Calculate the average time to read a single random sector.
2. A disk queue has requests for tracks: 55, 58, 39, 18, 90, 160, 150, 38, 184. Head starts at 50. Compute total head movement for FCFS and SSTF.
3. Explain why magnetic tape is still used for backups despite its slow access time.
4. A university stores student records in a file. The registrar needs to (a) print an alphabetical list of all students, (b) look up a student by matricule number. Which access method is most appropriate? Justify.
5. True/False: “SSDs have shorter seek time than HDDs because their tracks are closer together.” Correct the statement if false.

SOLUTION **Exercise 1**

1. Rotational latency = $\frac{30}{10\,000}$ = 3 ms. Transfer time = $\frac{4\times1024}{200\times10^6}$ ≈ 0.02 ms. Total ≈ 4+3+0.02 = 7.02 ms.
2. FCFS: 50→55(5)→58(3)→39(19)→18(21)→90(72)→160(70)→150(10)→38(112)→184(146) = 458. SSTF: 50→55(5)→58(3)→39(19)→38(1)→18(20)→90(72)→150(60)→160(10)→184(24) = 214.
3. Tape offers very low cost per GB, high capacity per cartridge, long archival life (30+ years), and offline protection against ransomware. Sequential access is acceptable for full backups.
4. Indexed sequential access: index on matricule for (b), sequential scan of index in alphabetical order (or secondary index on name) for (a).
5. False. SSDs have no seek time because they have no moving parts and no physical tracks/sectors. Access is electronic via LBA-to-physical-page mapping.

6 OS SECURITY AND ERROR MANAGEMENT

In the previous section we studied how files are physically stored on mass storage devices and how the operating system locates them using sequential, direct and indexed access methods. But storing data efficiently is not enough: the data must also be protected. A school's examination results stored on a hard disk are useless if any student can modify them, if a virus erases them, or if the disk fails the day before printing. This final section therefore studies how the operating system guarantees the **security** of files and devices, and how it **detects and handles errors** so that the whole system remains reliable.

6.1 Goals of Operating System Security

Imagine the computer room of a lycée in Yaoundé. The bursar's spreadsheet of school fees, the teachers' mark sheets and the students' own projects all live on the same machines. Three questions immediately arise: Who is allowed to see a file? Who is allowed to change it? And will the file still be there tomorrow? These three questions define the three classical goals of OS security.

DEFINITION **The CIA goals of OS security**

An operating system protects files and devices according to three goals, often called the CIA triad:

- **Confidentiality**: only authorised users can read a file or use a device. A student must

not be able to open the file containing the GCE mock examination marks.

- **Integrity**: only authorised users can modify a file, and the data must not be corrupted accidentally. A mark, once validated by the teacher, must not be altered.
- **Availability**: files and devices must be accessible when needed by legitimate users. A file destroyed by a disk crash or a virus violates availability just as surely as a thief violates confidentiality.

PROPERTY **Which mechanism protects which goal**

Permissions (access rights) protect confidentiality and integrity, because they control who may read and who may write. **Backups** protect availability, because they allow lost or damaged data to be restored. A complete security policy therefore needs both: permissions without backups leave you defenceless against hardware failure, and backups without permissions leave you defenceless against malicious users.

6.2 Authentication: Proving Who You Are

Before the operating system can decide what a user may do, it must know who the user is. This is the role of authentication.

DEFINITION **Authentication**

Authentication is the process by which a user proves his or her identity to the operating system before being granted access to the system. It is usually based on one or more of three factors:

- something the user knows: a **password** or a **PIN** (Personal Identification Number);
- something the user has: a smart card, a security token, a mobile phone;
- something the user is: a **biometric** feature such as a fingerprint, the face or the iris of the eye.

Combining two factors (for example a password and a code sent to a phone) is called two-factor authentication and is much stronger than a password alone.

Once authenticated, the user works under a **user account**. The operating system maintains, for each account, a **user identifier** (UID), a home directory and a record of the **groups** to which the user belongs. A **group** is a named collection of users who share the same needs: in a school server one might create the groups `teachers`, `students` and `admin`. Managing rights per group instead of per person is far easier: when a new teacher arrives, the administrator simply adds her to the `teachers` group and she instantly inherits all its permissions. The most powerful account, called the **administrator** (or `root` under Linux), can create and delete accounts, install

software and change any permission; ordinary accounts cannot.

WARNING **Weak passwords destroy everything**

Authentication is only as strong as the password. Passwords such as `12345`, `password` or a date of birth are guessed in seconds. A good password is long (at least 8–12 characters), mixes letters, digits and symbols, and is never written on a paper stuck to the monitor. Sharing one's password with a classmate breaks confidentiality completely, because every action done with your account will be attributed to you.

6.3 File Protection: Access Rights and Access Control Lists

Authentication answers "who are you?". **Authorisation** answers "what may you do?". The operating system attaches to every file and directory a set of **access rights** describing which operations are permitted.

DEFINITION **Access rights and access control list**

The basic **access rights** on a file are:

- Read (r): open the file and view or copy its contents;
- Write (w): modify, append to or delete the file;
- Execute (x): run the file as a program (for a directory, `x` means the right to enter it and access its entries).

An **access control list** (ACL) is the list, attached to a file, that specifies for each user or group which of these rights are granted or denied. When a process asks to open a file, the operating system compares the identity of the user with the file's ACL and either grants or refuses the operation.

Linux (and UNIX-like systems) implement a compact form of ACL. Every file has three categories of users: the **owner** (u), the **group** (g) and **others** (o). For each category, three rights (r, w, x) are either present (letter shown) or absent (replaced by a dash). The command `ls -l` displays them as a ten-character string, for example `rwxr-x---`.

METHOD **Reading Linux-style permissions**

To decode a permission string such as `rwxr-x---`:

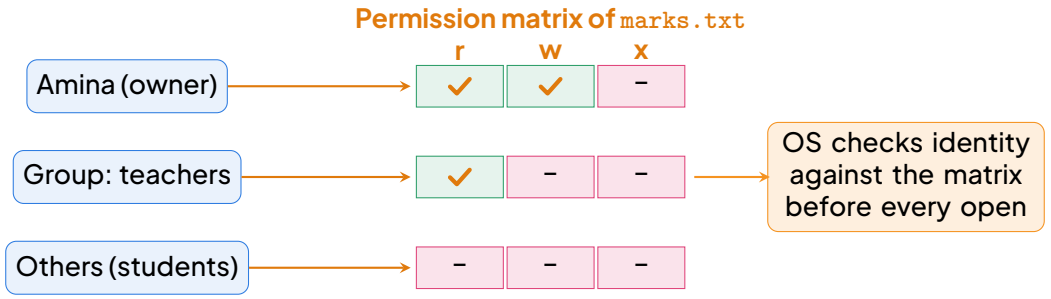
1. Split the nine characters into three blocks of three: `rwx` | `r-x` | `---`.
2. The first block gives the rights of the owner; the second block those of the group; the third block those of all other users.

3. Inside each block, the positions are always read, write, execute: a letter means the right is granted, a dash - means it is refused.
4. Conclude in plain English. Here: the owner may read, write and execute; group members may read and execute but not write; everybody else may do nothing at all.

Each right also has a numeric value (read = 4, write = 2, execute = 1); adding them per block gives the octal form, e.g. `rwxr-x---` = 750.

EXEMPLE **Decoding a permission string**

A teacher stores the file `marks.txt` with permissions `rw-r-----`. Block 1 (`rw-`): the owner (the teacher) can read and write, not execute. Block 2 (`r--`): members of the group `teachers` can only read. Block 3 (`---`): students, who are "others", cannot even open the file. Confidentiality of the marks is preserved, and integrity is preserved because nobody except the owner can write.



Access control: each user category is matched against the file's permission matrix; ✓ = granted, – = refused.

6.4 Threats to Files, Backups and Recovery

Even with perfect permissions, files face many threats. The main ones are:

- **Accidental deletion or modification**: a tired user deletes the wrong folder; a program crashes while saving and leaves a half-written file.
- **Malware**: viruses, worms and ransomware (which encrypts the victim's files and demands payment) destroy confidentiality, integrity and availability at once.
- **Unauthorised access**: a stolen password, or a computer left unlocked, lets an intruder read or alter private files.
- **Hardware failure**: a worn-out hard disk develops bad sectors and whole files become unreadable.

The fundamental defence against loss of availability is the **backup**: a copy of important files kept on a different device (external disk, server, cloud storage). If the original is destroyed, the backup is **restored**. Good backup policy follows three rules: backups must be **regular** (a backup from last year is of little use), **stored separately** from the original (a backup on the same disk dies with the disk), and **tested** (an untested backup may be unreadable the day it is needed). Many systems also keep a recycle bin and file version history, which allow recovery from simple accidental deletion without touching the backups.

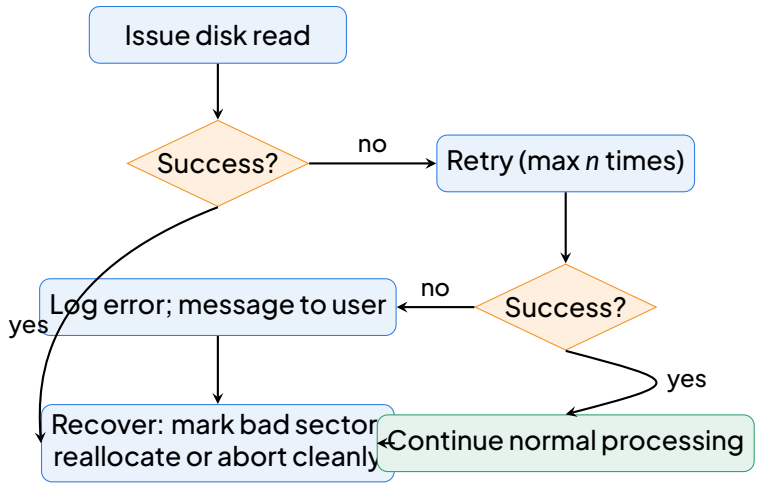
DID YOU KNOW? **Ransomware in everyday life**

Ransomware attacks have struck hospitals, town councils and schools around the world, encrypting all their files and demanding payment to unlock them. Organisations that had recent, separate backups simply wiped the infected machines and restored their data without paying. Those without backups sometimes lost years of records. This is why teachers insist: backups protect availability.

6.5 I/O Error Detection and Handling

Security protects data from people; error management protects data from hardware and software failures. When the operating system issues an I/O command — say, "read block 512 from the disk" — many things can go wrong: the device may be unplugged, the disk surface may have a **bad sector** (a physically damaged area that no longer holds data reliably), or the device may simply be busy. A robust operating system never assumes success: every I/O operation returns a **status code** that the OS examines.

The classical error-handling strategy proceeds in stages. First, the OS **retries** the operation a few times, because many errors are transient (a read head slightly misaligned may succeed on a second attempt). If retries fail, the OS **reports** the error: it logs it for the administrator and displays a clear **error message** to the user ("Disk not ready", "File could not be read"), rather than crashing silently. Finally, it tries to **recover**: it may mark the bad sector so that it is never used again, reallocate the data to a spare sector, or abort the operation cleanly while preserving the rest of the system.



OS error handling when a disk read fails: retry, then report, then recover.

6.6 System Reliability: Journaling, RAID and Safe Removal

Beyond reacting to errors, modern systems are designed to survive them.

Journaling. Suppose power fails while the OS is in the middle of writing a file: the file's data block has been updated but its directory entry has not — the file system is now inconsistent. A **journaling file system** (such as NTFS or ext4) first writes a description of the intended operation into a special log, the **journal**, and only then performs the operation. After a crash, the OS simply replays or cancels the operations recorded in the journal, restoring consistency in seconds instead of scanning the whole disk.

Disk mirroring (RAID). The idea behind **RAID** (Redundant Array of Independent Disks) is to use several disks so that the failure of one does not destroy data. In the simplest form, **mirroring** (RAID 1), every block is written identically to two disks; if one disk dies, the other still holds everything and the system keeps running while the failed disk is replaced. Other RAID levels spread data and error-correcting parity information across several disks to combine speed with protection.

Safe removal. For performance, the OS keeps written data temporarily in a cache in main memory and copies it to the USB drive a little later. If the user pulls the drive out during that delay, part of the file never reaches the disk.

WARNING **Never unplug a USB drive during a write**

Removing a USB flash drive while a write operation is still in progress — or before using "Eject" / "Safely remove hardware" — can leave the file half-written and corrupt the whole file system of the drive, making every file on it unreadable. Always close files on the drive, then use the safe-removal command and wait for the confirmation message before unplugging.

6.7 Good Practices for a School Computer Room

All these mechanisms only work if people use them. For a school computer room, the following policy summarises this section:

- **One account per user**, organised into groups (`students`, `teachers`); no shared accounts, no working as administrator for everyday tasks.
- **Correct permissions** on sensitive folders: marks and financial files readable only by authorised staff.
- **Regular, separate backups** of important files, tested from time to time.
- An up-to-date **antivirus**, and regular **system updates**, since updates repair the security holes that malware exploits.
- **Safe removal** of USB drives, and machines shut down properly rather than switched off at the wall.

WORKED EXAMPLE 5 **Choosing and justifying permissions**

On the school server, the file `gce_marks.csv` belongs to Mrs Ekane, and the group `teachers` must be able to consult it. Students must not open it. (a) Propose a Linux permission string. (b) Justify each block. (c) Explain why permissions alone are not sufficient protection.

SOLUTION

(a) A suitable string is `rw-r-----` (octal 640).
(b) Block 1, `rw-`: the owner, Mrs Ekane, must read the marks and write (update) them; execute is useless for a data file. Block 2, `r--`: the other teachers may consult the file but must not modify it, which protects integrity. Block 3, `---`: students fall under "others" and receive no right at all, which protects confidentiality.
(c) Permissions only control who accesses the file; they cannot protect against hardware failure, accidental deletion by the owner herself, or ransomware running under an authorised account. To guarantee availability, the school must also make regular backups on a separate device; to guarantee integrity against malware it needs an antivirus and updates. Security = permissions plus backups plus good practices.

EXERCISE 6 **Permissions, threats and reliability**

1. Decode the permission string `rwxr-xr--`: state precisely what the owner, the group and others may do, and give its octal value.
2. For each of the following incidents, state which CIA goal is violated and which mechanism would have prevented or repaired the damage: (a) a virus encrypts all files on the secretary's PC; (b) a student reads the principal's file left in a shared folder with

permission `rw-rw-rw-`; (c) a power cut corrupts a file being saved.

3. Explain, in four or five lines, how a journaling file system and disk mirroring each contribute to reliability, and why a school might prefer one, the other, or both.

SOLUTION **Exercise 1**

1. `rwxr-xr--`: owner may read, write and execute; group may read and execute but not write; others may only read. Octal: 7, 4 + 1 = 5, 4, i.e. 754.
2. (a) Availability (and integrity) violated; regular separate backups would allow restoration, an antivirus and updates would have reduced the risk. (b) Confidentiality violated; correct permissions (`rw-r-----`) or proper group management would have prevented it. (c) Integrity violated; a journaling file system would restore consistency after the crash.
3. Journaling records intended operations in a log so that after a crash the file system can be returned to a consistent state quickly; it protects against interruptions but not against the physical death of the disk. Mirroring (RAID 1) keeps a full copy of the data on a second disk, so the system survives the hardware failure of one disk. The two are complementary: a school server ideally uses both, since journaling does not replace a disk and mirroring does not repair a corrupted file system — and neither replaces backups.

KEY POINTS **Key points of the section**

- OS security pursues three goals: **confidentiality**, **integrity** and **availability** of files and devices.
- **Authentication** (password, PIN, biometrics) identifies the user; **access rights** and **ACLs** (e.g. Linux `rwxr-x---`) control what each user and group may do.
- Permissions protect confidentiality and integrity; **backups** protect availability. Both are indispensable.
- The OS handles I/O errors by retrying, then reporting clearly, then recovering (bad-sector management, clean abort).
- **Journaling**, **RAID** mirroring and safe removal of USB drives keep the system reliable; in a school computer room these techniques must be backed by user accounts, antivirus, updates and regular backups.

7 INTEGRATION ACTIVITY

7.1 Aims of the activity

This integration activity is the heart of the integration activity: it is a guided design task in which you will apply the concepts of I/O management, file systems, storage and security concepts in one realistic design task. The school receives 20 computers, 2 printers and one server with a 1 TB disk. In groups, students must: (1) propose how the OS will manage the shared printers using spooling; (2) choose a file system for the server and justify it; (3) design the directory tree for staff and students with absolute paths; (4) choose an allocation/access strategy for the student records file that must be searched quickly by name; (5) define user accounts and permissions so students cannot read teachers' marks; (6) propose a backup and error-recovery plan in case of power cuts. Each group presents and defends its choices in a plenary session.

7.2 Situation

Setting up the computer room of GBHS Bamenda The school receives 20 computers, 2 printers and one server with a 1 TB disk. In groups, students must: (1) propose how the OS will manage the shared printers using spooling; (2) choose a file system for the server and justify it; (3) design the directory tree for staff and students with absolute paths; (4) choose an allocation/access strategy for the student records file that must be searched quickly by name; (5) define user accounts and permissions so students cannot read teachers' marks; (6) propose a backup and error-recovery plan in case of power cuts. Each group presents and defends its choices in a plenary session.

7.3 Tasks

The school receives 20 computers, 2 printers and one server with a 1 TB disk. In groups, students must: (1) propose how the OS will manage the shared printers using spooling; (2) choose a file system for the server and justify it; (3) design the directory tree for staff and students with absolute paths; (4) choose an allocation/access strategy for the student records file that must be searched quickly by name; (5) define user accounts and permissions so students cannot read teachers' marks; (6) propose a backup and error-recovery plan in case of power cuts. Each group presents and defends its choices in a plenary session.

7.4 Model answer

The school receives 20 computers, 2 printers and one server with a 1 TB disk. In groups, students must: (1) propose how the OS will manage the shared printers using spooling; (2) choose a file system for the server and justify it; (3) design the directory tree for staff and students with absolute paths; (4) choose an allocation/access strategy for the student records file that must be searched quickly by name; (5) define user accounts and permissions so students cannot read teachers' marks; (6) propose a backup and error-recovery plan in case of power cuts. Each group presents and defends its choices in a plenary session. Each group presents and defends its choices in a plenary session. Each group presents and defends its choices in a plenary session.

8 METHODS AND WORKED EXAMPLES

9 MANAGING INPUT/OUTPUT OPERATIONS AND FILE SYSTEMS

9.1 Method 1 — Identifying the Role of Each Component of the I/O Subsystem

METHOD **Analysing an I/O device management situation**

When a question describes how a computer communicates with a peripheral (keyboard, printer, disk, flash drive), proceed as follows:

1. Identify the device class: is it a **character device** (data as a stream of characters: keyboard, mouse, serial printer) or a **block device** (data in fixed-size blocks: hard disk, SSD, USB flash)?
2. Locate the device controller (adapter): the hardware circuit between the bus and the device. It contains registers (data, status, control) that the CPU reads/writes.
3. Name the software layers: user program → device-independent OS layer (system calls `read`, `write`) → **device driver** (device-specific code) → interrupt handler → hardware.
4. State the role of the driver: it translates generic OS requests into device-specific commands and must be installed for the device to work.
5. Mention buffering: the OS keeps a **buffer** in memory to absorb speed differences between fast CPU and slow devices (e.g. print spooling).

Reflex: "driver = translator, controller = hardware interface, buffer = shock absorber."

KEY POINTS **Key components of the I/O subsystem**

- Device controller: hardware interface with registers (status, control, data).
- Device driver: kernel module that knows the device's command set; translates logical requests into physical operations.
- Interrupt handler: routine executed when the device signals completion.
- Buffer / Spool: temporary storage that decouples producer (CPU) and consumer (device) speeds.

WORKED EXAMPLE 6 **Device management at a cybercafé in Buea**

A cybercafé in Buea buys a new USB printer. When connected, the OS displays "device not recognised". After installing software from the manufacturer's CD, the printer works. The owner notices that a 50-page document is sent to the printer in a few seconds, yet printing takes 10 minutes, and other users can continue working meanwhile.

a) What software was missing initially, and what is its role?

b) Is a printer a character or a block device? Justify.

c) Which I/O management technique allows users to continue working while printing? Explain.

SOLUTION

a) The missing software is the **device driver**. A driver is the device-specific program of the OS that translates generic operating-system commands ("print this data") into the exact control signals understood by that particular printer model. Without it, the OS can detect that something is plugged into the USB port but cannot communicate with it, hence "device not recognised".
b) A printer is a **character device**: it accepts and processes data as a sequential stream of characters (bytes), with no notion of fixed-size addressable blocks and no random access. By contrast, a hard disk is a block device because data is read/written in fixed-size blocks (sectors) that can be accessed in any order.
c) The technique is **spooling** (Simultaneous Peripheral Operations On-Line), a form of buffering. The OS copies the whole document quickly into a reserved area (the print queue/buffer) on disk or in memory. The application is then released and users continue their work, while a background process (the print spooler) feeds the data to the slow printer at the printer's own pace. The buffer absorbs the speed mismatch between the fast computer and the slow printer.

9.2 Method 2 — Choosing and Comparing I/O Transfer Mechanisms

METHOD **Programmed I/O, Interrupt-driven I/O, DMA**

To answer any question comparing the three transfer mechanisms:

1. Programmed I/O (polling): the CPU repeatedly tests the device status register in a loop until the device is ready. Simple, but the CPU is wasted waiting.
2. Interrupt-driven I/O: the CPU starts the transfer, then does other work; the device controller raises an **interrupt** when finished. The CPU suspends its program, runs the **interrupt handler**, then resumes.
3. DMA (Direct Memory Access): for large block transfers, the CPU gives the **DMA con-**

troller the memory address, the byte count and the direction; the controller moves the data directly between device and memory without the CPU, and interrupts only once at the end.

Key comparison to remember:

Criterion	Programmed I/O	Interrupt I/O	DMA
CPU involvement	Total (busy waiting)	Per byte/word	Start and end only
CPU efficiency	Very low	Medium	Very high
Best for	Very simple systems	Slow devices (keyboard)	Large blocks (disk)

Reflex: the more advanced the mechanism, the less the CPU is involved and the larger the suitable data volume.

WARNING **Common mistake**

Do not confuse interrupt-driven I/O with DMA. In interrupt-driven I/O, the CPU still moves each byte/word (via the ISR); in DMA, a separate controller moves the whole block while the CPU is free.

WORKED EXAMPLE 7 **Comparing CPU usage for a disk transfer**

A computer must transfer a 4 KB disk block into main memory. The disk controller can deliver one 32-bit word every 2 μs. In programmed I/O, testing the status register and copying one word costs the CPU 2 μs per word. With DMA, the CPU spends 10 μs initialising the DMA controller and 5 μs handling the single end-of-transfer interrupt.

a) How many words does the block contain?

b) Compute the total CPU time used with programmed I/O.

c) Compute the total CPU time used with DMA, and the percentage of CPU time saved.

d) State which mechanism is appropriate and why.

SOLUTION

a) Number of words:

$$N = \frac{4\,\text{KB}}{32\,\text{bits}} = \frac{4 \times 1024\,\text{bytes}}{4\,\text{bytes}} = 1024\,\text{words}.$$

b) Programmed I/O: the CPU is busy for the whole transfer of every word:

$$T_{\text{PIO}} = 1024 \times 2\,\mu s = 2048\,\mu s \approx 2.05\,\text{ms}.$$

During all this time the CPU does nothing useful (busy waiting).
c) DMA: the CPU is involved only at the start and at the end:

$$T_{\text{DMA}} = 10 + 5 = 15\,\mu s.$$

Percentage of CPU time saved:

$$\frac{2048 - 15}{2048} \times 100 = \frac{2033}{2048} \times 100 \approx 99.3\,\%.$$

d) DMA is appropriate: the transfer is a large block of data, and DMA frees the CPU for other tasks during the 2048 μs of actual data movement, using only 15 μs of CPU time (a saving of about 99.3 %). Programmed I/O would monopolise the CPU for the entire transfer.

9.3 Method 3 — Tracing Interrupt-Driven I/O

METHOD **Writing the sequence of events of an interrupt**

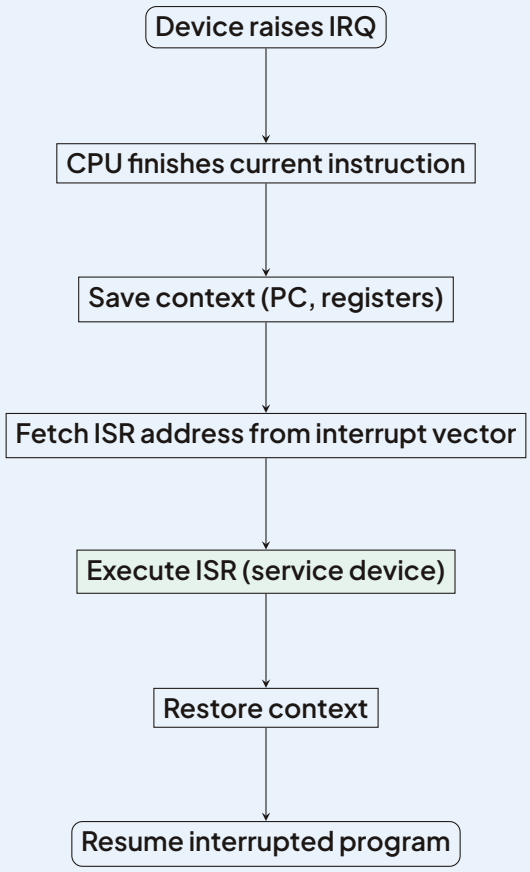
Exam questions often ask you to "describe what happens when a key is pressed" or "when the disk finishes a transfer". Always give the steps in this order:

1. The device controller completes its operation and sends an **interrupt request** (IRQ) on the bus.
2. The CPU finishes the current instruction, then saves the **context** (program counter, registers) of the running program.
3. The CPU uses the **interrupt vector** to find the address of the corresponding **interrupt service routine** (ISR / handler).
4. The ISR executes: it services the device (e.g. reads the character, clears the interrupt flag).
5. The saved context is restored and the interrupted program resumes exactly where it stopped.

Reflex: "finish instruction → save → jump to handler → serve → restore → resume."

EXPERIMENT **Interrupt handling flowchart**

The following flowchart summarises the interrupt cycle. It can be drawn in exams when a description is required.



WORKED EXAMPLE 8 **Interrupt handling for a keyboard**

A student types the letter “A” on a keyboard while a spreadsheet program is recalculating. Describe, in numbered steps, how the computer uses interrupt-driven I/O to capture the keystroke without stopping the recalculation permanently.

SOLUTION

1. The key press is converted by the keyboard controller into a scan code stored in its data register.
2. The keyboard controller sends an interrupt request to the CPU on its IRQ line.
3. The CPU completes the current machine instruction of the spreadsheet recalculation (interrupts are checked between instructions).
4. The CPU saves the context of the spreadsheet program: the program counter and the working registers are pushed onto the stack.
5. Using the interrupt number, the CPU consults the interrupt vector table and jumps to

the keyboard interrupt service routine.

6. The ISR reads the scan code from the keyboard controller's data register, converts it to the character "A", places it in the keyboard buffer, and resets the controller's interrupt flag.
7. The CPU restores the saved registers and program counter from the stack.
8. The spreadsheet recalculation resumes from the exact point where it was suspended; the character "A" waits in the buffer until the program reads it.

The recalculation was only paused for the few microseconds of the ISR, not stopped: this is the whole advantage of interrupt-driven I/O over polling.

9.4 Method 4 — Computing Disk Access Time and Applying Scheduling Algorithms

METHOD **Disk access time and head scheduling**

Key formula:

$$\text{Access time} = \text{seek time} + \text{rotational latency} + \text{transfer time}.$$

1. Seek time: time to move the read/write head to the correct **track** (cylinder). Often given as a time per track crossed or an average value.
2. Rotational latency: time for the desired **sector** to rotate under the head; on average half a revolution: $t_{rot} = \frac{1}{2} \times \frac{60}{N}$ s for N revolutions per minute.
3. Transfer time $= \frac{\text{data size}}{\text{transfer rate}}$.

For scheduling, list requests, then simulate:

- FCFS: serve in arrival order.
- SSTF: always serve the pending request nearest to the current head position.
- SCAN (lift algorithm): move in one direction serving requests, reverse at the end.
- C-SCAN: move in one direction only, jump back to start without serving.

Total head movement = sum of absolute differences between successive positions.

KEY POINTS **Disk access time components**

- Seek time dominates for random access; scheduling reduces it.

- Rotational latency averages half a rotation; for 7200 rpm ≈ 4.17 ms.
- Transfer time is usually negligible for small blocks (e.g. 4 KB at 100 MB/s ≈ 0.04 ms).

WORKED EXAMPLE 9 **Access time and scheduling on a hard disk**

A disk rotates at 7200 rpm. Its average seek time is 8 ms and its transfer rate is 100 MB/s.

a) Compute the average time to read a 4 KB block.

b) The head is at track 50 and the queue of requests is: 82, 40, 65, 20, 95. Compute the total head movement with FCFS, then with SSTF.

SOLUTION

a) Average access time:

$$t_{seek} = 8\ \text{ms}$$

$$t_{rot} = \frac{1}{2} \times \frac{60}{7200}\ \text{s} = \frac{1}{240}\ \text{s} \approx 4.17\ \text{ms}$$

$$t_{transfer} = \frac{4\ \text{KB}}{100\ \text{MB/s}} = \frac{4 \times 10^3}{100 \times 10^6}\ \text{s} = 0.04\ \text{ms}$$

$$T = 8 + 4.17 + 0.04 \approx 12.21\ \text{ms}.$$

Note that the seek time dominates: this is why head scheduling matters.

b) FCFS from track 50, order 82, 40, 65, 20, 95:

Move	Tracks crossed	Cumulative
50 → 82	32	32
82 → 40	42	74
40 → 65	25	99
65 → 20	45	144
20 → 95	75	219

Total FCFS movement = 219 tracks.

SSTF from track 50: nearest requests are chosen successively:

Head at	Nearest pending	Tracks crossed	Cumulative
50	40	10	10
40	20	20	30
20	65	45	75
65	82	17	92
82	95	13	105

Total SSTF movement = 105 tracks, less than half of FCFS. SSTF reduces average seek time, but a request far from the head may wait a long time (risk of **starvation**).

9.5 Method 5 — Tracing File Allocation (Contiguous, Linked, Indexed, FAT)

METHOD **Simulating file allocation methods**

Given a disk divided into blocks and a file of a given size:

1. Compute the number of blocks needed: $n = \left\lceil \frac{\text{file size}}{\text{block size}} \right\rceil$.
2. Contiguous allocation: the directory stores (start block, length); all blocks are consecutive. Fast, but suffers from **external fragmentation**.
3. Linked allocation: each block holds a pointer to the next; the directory stores the first block. No external fragmentation, but random access is slow (must follow the chain).
4. Indexed allocation: an **index block** contains the addresses of all the file's blocks (like the inode in UNIX). Direct access is fast; the index block costs one extra block.
5. FAT: a table with one entry per disk block; each entry gives the next block of the file, or an end-of-file (EOF) mark. To trace a file, follow the chain in the FAT from the first block given in the directory.

Reflex: always draw the block chain and follow pointers step by step; never guess.

EXEMPLE **Indexed allocation (UNIX inode)**

A file of 12 blocks uses an inode with 10 direct pointers, 1 single indirect, 1 double indirect, 1 triple indirect. The index block for single indirect holds 256 pointers (if block size = 4 KB and pointer = 4 bytes). This allows huge files while keeping small files efficient (no extra indirection).

WORKED EXAMPLE 10 **Tracing a file in a FAT-structured disk**

A disk uses a File Allocation Table. Block size is 1 KB. The directory entry of file `NOTES.TXT` gives first block = 5. The relevant FAT entries are:

Block	2	3	5	7	8	9	11	12
FAT entry	9	8	7	12	EOF	3	2	11

a) List, in order, the blocks occupied by `NOTES.TXT`.

b) The file `EXAM.DOC` starts at block 9. List its blocks.

c) What is the size in bytes of the largest file that can be stored in the blocks used by `NOTES.TXT`?

d) Give one advantage and one disadvantage of FAT (linked) allocation compared with contiguous allocation.

SOLUTION

a) Follow the chain from block 5:

5 → FAT[5] = 7 → FAT[7] = 12 → FAT[12] = 11 → FAT[11] = 2 → FAT[2] = 9 → FAT[9] = 3 → FAT[3] = 8 → FAT[8

Blocks of `NOTES.TXT`: 5, 7, 12, 11, 2, 9, 3, 8 (8 blocks).
b) From block 9: 9 → 3 → 8 → EOF. Blocks of `EXAM.DOC`: 9, 3, 8.
Remark: in a real system two files cannot share blocks; the data given here is a simplified exercise, and the trace method is what matters.
c) 8 blocks of 1 KB = 8 × 1024 = 8192 bytes. (The last block may be partially filled, so the actual file is between 7169 and 8192 bytes.)
d) Advantage: no external fragmentation — any free block anywhere on the disk can be used, so files can grow easily and no compaction is needed. Disadvantage: random access is slow: to reach block number k of the file, the chain must be followed through k FAT entries, whereas contiguous allocation computes the address directly (start $+k$). The FAT itself must also be kept in memory, consuming RAM.

9.6 Method 6 — Applying File Access Methods and Directory Operations

METHOD **Choosing a file access method and computing record position**

1. Sequential access: records are read in order, one after another (like a cassette tape). Suitable for batch processing (e.g. payroll for all workers of a company).
2. Direct (random) access: any record can be reached immediately by its position or key. For fixed-length records:

   $$\text{offset of record } k = (k - 1) \times \text{record length}.$$

   Suitable for databases, bank account enquiries.
3. Indexed sequential (ISAM): an index gives the approximate location, then a short sequential search finishes. Good compromise for files processed both ways.

Reflex: ask “does the application read all records in order, or one particular record quickly?” — the answer chooses the method.

KEY POINTS **File access methods summary**

Method	Best for	Typical use
Sequential	Processing all records in order	Payroll, log files, backups
Direct (random)	Accessing a single record by key	Database queries, airline reservations
Indexed sequential	Mixed workload	Library catalogues, student records

WORKED EXAMPLE 11 **Locating a student's record in a results file**

A school in Yaoundé stores GCE candidate results in a file of fixed-length records. Each record occupies 128 bytes. Records are stored consecutively from byte 0, ordered by candidate number, starting at candidate 1.

a) Using direct access, at what byte offset does the record of candidate number 350 begin?

b) How many disk blocks of 512 bytes must be read to fetch this record, and which blocks (numbered from 0)?

c) The school also produces an end-of-year report that processes every candidate in candidate-number order. Which access method is best for this task, and why?

SOLUTION

a) Offset of record k:

$$\text{offset} = (k - 1) \times 128 = (350 - 1) \times 128 = 349 \times 128 = 44\,672 \text{ bytes.}$$

b) Block containing byte 44 672:

$$\left\lfloor \frac{44\,672}{512} \right\rfloor = \lfloor 87.25 \rfloor = 87.$$

The record spans bytes 44 672 to 44 799. Byte 44 799 lies in block $\lfloor 44\,799/512 \rfloor = 87$ as well ($87 \times 512 = 44\,544$; $88 \times 512 = 45\,056$). So the record fits entirely in block 87: only 1 block must be read.

c) Sequential access is best: the report reads every record in candidate-number order, which is exactly the physical order of the file. Sequential reading is then the fastest possible strategy — each block is read once, in order, with no repeated index lookups or head movements. Direct access would add useless address computations for a task that touches all records anyway.

9.7 Method 7 — Managing OS Security: Permissions and Access Control

METHOD **Reading and setting file permissions**

1. Identify the three categories of users: **owner** (u), **group** (g), **others** (o).
2. Identify the three rights: **read** (r = 4), **write** (w = 2), **execute** (x = 1).
3. To convert symbolic to octal: add the values in each category. Example: `rwxr-x---` = (4+2+1)(4+0+1)(0) = 750.
4. To convert octal to symbolic: expand each digit in binary over 3 bits: 6 = 110 = `rw-`.
5. For an **access control matrix**: rows = users, columns = objects (files), cells = allowed operations. To answer "can user U do operation X on file F?", read the cell (U, F).
6. Remember the security goals: **confidentiality**, **integrity**, **availability**; and the protection tools: authentication (passwords), authorisation (permissions), encryption, backups.

WARNING **Common permission pitfalls**

- `chmod 777` gives everyone full access — a major security risk.
- The `execute` bit on a directory means "traverse" (cd into it); without it, you cannot access files inside even if you can read them.
- Changing group ownership (`chgrp`) may be needed before granting group permissions.

WORKED EXAMPLE 12 **Permissions on a school server**

On the Linux server of a college in Bamenda, the file `marks.csv` containing examination marks has permissions shown by the listing:

```
-rw-r----- 1 mr.tanyi teachers 20480 Jun 12 09:30 marks.csv
```

a) Interpret these permissions for the owner, the group and the others.

b) Give the octal (numeric) equivalent of these permissions.

c) The principal wants the secretary (who is not in the `teachers` group) to be unable to read the marks, and Mr Tanyi to be able to modify them. Are the current permissions correct? Justify.

d) State the command that would additionally give the group write permission.

SOLUTION

a) The permission string is `rw-r-----`, split into three triplets:

- Owner (`mr.tanyi`): `rw-` ⇒ read and write, no execute.
- Group (`teachers`): `r--` ⇒ read only.
- Others: `---` ⇒ no right at all.

b) Octal conversion:

$$\texttt{rw-} = 4 + 2 = 6, \qquad \texttt{r--} = 4, \qquad \texttt{---} = 0 \;\Rightarrow\; 640.$$

c) Yes, the current permissions are correct for both requirements: the secretary belongs to "others", whose rights are `---`, so she can neither read nor modify the file (confidentiality respected); the owner Mr Tanyi has `w`, so he can modify the marks.
d) The command is:

```
chmod g+w marks.csv
```

This changes the permissions from `rw-r-----` (640) to `rw-rw----` (660). Note that this would let every teacher modify the marks — a decision the principal should weigh carefully, since it affects the **integrity** of the results.

9.8 Method 8 — Diagnosing Errors and Applying OS Error Management

METHOD **Classifying errors and choosing a recovery strategy**

1. Classify the error: hardware (disk failure, power cut), software (division by zero, invalid memory access), user (wrong password, deleting a file), I/O (printer out of paper, device not responding).
2. Detection mechanisms: status codes returned by devices, parity bits and checksums for data corruption, timeouts for non-responding devices, exception signals (interrupts) for runtime faults.
3. Handling strategies, in increasing order of severity:
   - Retry the operation (transient errors, e.g. re-read a disk sector);
   - Report to the user with a clear error message (e.g. "Printer offline");
   - Abort the faulty process while protecting the rest of the system;
   - Recover using backups, journaling file systems or restore points after a crash.
4. Prevention: backups, UPS against power cuts, user permissions, validation of input

data.

Reflex: a good answer always follows the chain detect → report → handle → recover → prevent.

DID YOU KNOW? **Journaling file systems in Cameroon**

In regions with unstable power supply (common in many parts of Cameroon), journaling file systems (ext4, NTFS, XFS) are essential. They keep a log (journal) of pending metadata changes. After a crash, the OS replays the journal to restore consistency in seconds, avoiding the long `fsck` scans of older systems. This dramatically reduces downtime in school computer labs and cybercafés.

WORKED EXAMPLE 13 **Error management scenario in a school computer room**

During the computerised entry of GCE marks, the following incidents occur in one morning:

(i) A teacher types a mark of 117 out of 100 and the program refuses it.

(ii) While saving, the program displays "Disk full — unable to save".

(iii) A power cut stops all the machines; on restart, the marks entered in the morning are recovered up to the last save, thanks to the journaling file system.

For each incident: classify the error, state how the OS/program detected it, and describe the handling or recovery mechanism illustrated.

SOLUTION

(i) Mark of 117/100 — user (data validation) error. Detection: the application performs input validation (range check $0 \leq \text{mark} \leq 100$) before accepting the value. Handling: the program rejects the input and reports a clear message, inviting correction. This protects the **integrity** of the data file. Prevention: validation rules built into the data-entry form.
(ii) "Disk full" — I/O / resource error. Detection: when the program issues the `write` system call, the file system finds no free block; the device-independent I/O layer returns a status/error code to the application, which displays the message. Handling: the operation is aborted without corrupting the existing file (a well-written program writes to a temporary file first, or keeps the data in memory). Recovery: the user frees disk space (deletes unneeded files, empties the recycle bin) or saves to another volume (USB flash drive), then retries the save.
(iii) Power cut — hardware/environmental failure. Detection: on reboot, the OS checks the file system and finds it was not cleanly unmounted. Recovery: the **journaling file system** replays its journal (log of recent write operations) to bring the file system back to a

consistent state, so all work up to the last completed save is preserved. Prevention: a UPS (uninterruptible power supply) would let users save and shut down properly during cuts — a very relevant precaution in areas with unstable electricity supply.
Conclusion: the three incidents illustrate the full error-management chain — detection (validation, status codes, journal check), reporting (clear messages), handling (rejection, safe abort) and recovery (journal replay) — showing how the OS protects both the data and the system from failures.

9.9 Integration Activity — Lesson 61

EXERCISE 7 **Comprehensive I/O and file system scenario**

A secondary school in Douala uses a Linux server to manage student records. The server has a 7200 rpm hard disk (average seek 8 ms, transfer rate 100 MB/s, block size 4 KB). Student records are fixed-length (256 bytes) and stored in a file `students.dat`. The file uses indexed allocation (inode with 12 direct pointers, then single/double/triple indirect). The directory entry for `students.dat` shows permissions `-rw-rw----` (owner: admin, group: staff). The school's backup script runs nightly via `cron`.
Answer the following:

a) Calculate the average disk access time to read one 4 KB block.

b) If the file currently occupies 150 blocks, how many index blocks are needed (assume 4-byte pointers, 4 KB blocks)?

c) A teacher (member of `staff`) tries to run a Python script that opens `students.dat` for writing. The script fails with "Permission denied". Explain why, and give the exact `chmod` command to fix it without compromising security.

d) During a thunderstorm, a power cut occurs while the backup script is writing to a USB drive. The USB drive uses FAT32 (no journaling). What is the likely state of the backup file after reboot? What should the administrator do?

e) The school wants to replace the hard disk with an SSD. Name two I/O management aspects that will change and two that will stay the same.

SOLUTION **Exercise 1**

a) Same as Method 4 example: t_{seek} = 8 ms, t_{rot} = 4.17 ms, $t_{transfer}$ = 0.04 ms ⇒ ≈ 12.21 ms.
b) Each index block holds 4096/4 = 1024 pointers. Direct pointers cover 12 blocks. Remaining 150 - 12 = 138 blocks need single indirect: one index block can cover 1024 blocks, so only 1 single indirect block is needed. No double/triple indirect required.

c) Permissions `rw-rw----` give group write? Wait: `rw-rw----` means owner rw, group rw, others none. The teacher is in group `staff`, so group has write. The error "Permission denied" suggests the teacher is not in the `staff` group, or the file's group is not `staff`. Check with `groups teacher`. If the teacher is not in the group, add them: `usermod -aG staff teacher`. If the group is wrong, change it: `chgrp staff students.dat`. Do not use `chmod o+w` (would open to everyone).
d) FAT32 has no journaling. The backup file may be corrupted (partial clusters, broken chain). The administrator should verify the backup (checksum), and if corrupted, re-run the backup from the source (which is on the journaling ext4 partition, so source data is safe). Future: use a journaling file system on backup media (e.g. ext4 on USB) or use a backup tool that writes atomically.
e) Changes: (1) Seek time becomes negligible (no moving head), so scheduling algorithms like SSTF/SCAN are irrelevant; (2) Wear levelling and TRIM become important for SSD longevity. Stays the same: (1) Block-based I/O, file system abstraction (inodes, directories); (2) Interrupt/DMA mechanisms for data transfer.

10 EXERCISES

10.1 I check my knowledge

EXERCISE 8 **MCQ: I/O Transfer Mechanisms**

Which of the following I/O transfer mechanisms allows the CPU to perform other tasks while data is being transferred between memory and an I/O device without CPU intervention for each byte?

1. Programmed I/O
2. Interrupt-driven I/O
3. Direct Memory Access (DMA)
4. Polling

EXERCISE 9 **True/False with Justification**

State whether each statement is true or false. Justify your answer briefly.

1. A device driver is a hardware component that connects an I/O device to the system bus.
2. Spooling allows multiple processes to share a single printer by queuing print jobs on disk.
3. In contiguous file allocation, a file can grow dynamically without any risk of external fragmentation.
4. The average rotational latency of a disk rotating at 7200 rpm is 4.17 ms.

EXERCISE 10 **Definitions**

Define the following terms precisely:

1. Buffering
2. Spooling
3. File descriptor (or file handle)
4. Access control list (ACL)

EXERCISE 11 **Direct Calculation: Disk Access Time**

A magnetic disk has the following characteristics: average seek time = 8 ms, rotation speed = 7200 rpm, transfer time for a 4 KB block = 0.1 ms. Compute the average access time to read one block. (Assume average rotational latency is half the rotation period.)

10.2 I practise

EXERCISE 12 **Comparison of I/O Transfer Mechanisms**

Complete the table below comparing programmed I/O, interrupt-driven I/O, and DMA. For each criterion, give a concise answer.

Criterion	Programmed I/O	Interrupt-driven I/O	DMA
CPU involvement during transfer			
Speed / Efficiency			
Typical example device			
Complexity of implementation			

EXERCISE 13 **File Access Methods**

A file contains 1000 records of fixed length. The application needs to:

1. Read records sequentially from the beginning to the end.
2. Occasionally jump directly to record number 500 and read a few records around it.
3. Insert a new record at the end of the file.

For each requirement, state the most suitable file access method (sequential, direct/relative, indexed sequential) and explain why.

EXERCISE 14 **Directory Tree and Paths**

Draw the directory tree for a school system with the following structure:

- Root directory: `/school`
- Under `/school`: directories `classes`, `staff`, `admin`
- Under `classes`: directories `form1`, `form2`, `form3`

- Under each form: directories for subjects `math`, `english`, `physics`
- Under each subject: files for students, e.g., `john.txt`, `mary.txt`

Give the absolute path of the file `john.txt` in `math` for `form2`.

EXERCISE 15 **File Allocation Methods**

A file of 10 blocks is stored on disk using three different allocation methods: contiguous, linked, and indexed (with a single index block). For each method, state:

1. The maximum number of disk accesses required to read the 8th block, assuming the directory entry is already in memory.
2. One advantage and one drawback for this specific file size.

10.3 I prepare for the GCE

EXERCISE 16 **Structured Question: Device Management and Spooling**

(15 marks)

1. Define the following terms: (3 marks)
   (a) Device driver
   (b) Buffering
   (c) Spooling
2. Explain how spooling allows three students to send print jobs to a single shared printer at the same time. Use a diagram or description of the spool directory and the print daemon. (6 marks)
3. The printer jams while printing the second student's job. Describe the steps the operating system's spooler should take to recover and ensure fairness. (6 marks)

EXERCISE 17 **Scenario: Disk Scheduling and File Corruption**

(15 marks) A disk has 200 cylinders (0–199). The current head position is at cylinder 50, moving towards higher cylinders. The pending requests (in order of arrival) are for cylinders: 82, 170, 43, 140, 24, 16, 190.

1. Calculate the total head movement (in cylinders) for the following disk scheduling al-

gorithms: (9 marks)

(a) FCFS (First-Come, First-Served)

(b) SSTF (Shortest Seek Time First)

(c) SCAN (Elevator)

2. A student copies a large file to a USB flash drive. During the copy, a power cut occurs. Afterwards, the file on the flash drive is corrupted. Explain why this happens in terms of file system metadata and data blocks. (3 marks)
3. Propose two mechanisms or good practices (at the OS or user level) that reduce the risk of such corruption. (3 marks)

EXERCISE 18 **Essay Question: OS Security and File Permissions**

(15 marks)

1. Interpret the Linux permission string `rwxr-x---` for a file named `grades.txt`. State exactly which categories of users (owner, group, others) can read, write, and execute the file. (4 marks)
2. The school's system administrator wants to allow teachers to read and write `grades.txt`, but students should only read it. The file is owned by `admin` and belongs to group `teachers`. Propose the exact `chmod` command (using symbolic or octal notation) to achieve this. (3 marks)
3. Explain the difference between authentication and authorisation in the context of operating system security. Give one example of each mechanism used in modern OSs. (4 marks)
4. A malicious program attempts to overwrite the system's password file. Describe two OS-level protection mechanisms that would prevent this. (4 marks)

11 SOLUTIONS TO THE EXERCISES

12 SOLUTIONS TO CHAPTER CSC12 EXERCISES

12.1 Solutions — I check my knowledge

SOLUTION **Exercise 1 — MCQ: I/O Transfer Mechanisms**

Correct answer: (c) Direct Memory Access (DMA).
Justification: In DMA, a dedicated DMA controller takes charge of the whole transfer between the I/O device and main memory. The CPU only initiates the transfer (it gives the controller the memory address, the number of bytes and the direction) and is then free to execute other instructions. The controller transfers entire blocks of data and sends a single interrupt to the CPU when the transfer is complete.
Why the other options are wrong:

- Programmed I/O: the CPU executes a loop that reads/writes every byte and constantly checks the device status — total CPU involvement.
- Interrupt-driven I/O: the CPU is freed between bytes, but it is still interrupted to handle each byte (or small unit) of the transfer.
- Polling: this is not a separate mechanism; it is the busy-waiting technique used by programmed I/O, so it also monopolises the CPU.

SOLUTION **Exercise 2 — True/False with Justification**

1. FALSE. A device driver is a software component (a program or kernel module), not hardware. It translates the OS's generic I/O commands into the specific commands understood by a particular device. The hardware component that physically connects a device to the system bus is the device controller (or interface adapter).
2. TRUE. Spooling (Simultaneous Peripheral Operations On-Line) stores print jobs in a queue on disk (the spool directory). A print daemon sends them one after another to the printer, so several processes can "print" simultaneously without waiting for the slow device.
3. FALSE. With contiguous allocation a file occupies one single run of consecutive blocks. It cannot grow if the neighbouring blocks are occupied, and repeated creation/deletion of files produces external fragmentation (free space scattered in small

unusable holes). Contiguous allocation is precisely the method most exposed to external fragmentation.

4. TRUE. At 7200 rpm, one full rotation takes $\frac{60\text{ s}}{7200} = \frac{60\,000\text{ ms}}{7200} \approx 8.33\text{ ms}$. The average rotational latency is half a rotation: $\frac{8.33}{2} \approx 4.17\text{ ms}$.

SOLUTION **Exercise 3 — Definitions**

1. Buffering: the use of a temporary storage area in main memory (a **buffer**) to hold data while it is being transferred between a process and an I/O device, in order to compensate for the speed difference between the fast CPU/memory and the slow device, and to allow the producer and the consumer of the data to work at different rates.
2. Spooling: a technique in which jobs destined for a dedicated, non-shareable device (typically a printer) are first written to a queue on disk (the spool area) and then sent to the device one at a time by a daemon, so that several processes can use the device "simultaneously" without conflict.
3. File descriptor (file handle): a small integer (or identifier) returned by the OS to a process when it opens a file; it is an index into the process's open-file table and is used in all subsequent operations (read, write, close) to refer to that file without repeating its name.
4. Access control list (ACL): a list attached to an object (file, directory) that specifies, for each user or group of users, the operations (read, write, execute, delete, …) that are allowed or denied on that object. The OS checks the ACL on every access request.

SOLUTION **Exercise 4 — Direct Calculation: Disk Access Time**

The average access time is the sum of three components:

$$T_{\text{access}} = T_{\text{seek}} + T_{\text{rotational latency}} + T_{\text{transfer}}$$

Step 1 — Rotation period. At 7200 rpm:

$$T_{\text{rotation}} = \frac{60\text{ s}}{7200} = 8.33\text{ ms}$$

Step 2 — Average rotational latency (half a rotation):

$$T_{\text{latency}} = \frac{8.33}{2} = 4.17\text{ ms}$$

Step 3 — Total:

$$T_{\text{access}} = 8\ \text{ms} + 4.17\ \text{ms} + 0.1\ \text{ms}$$
$$= \boxed{12.27\ \text{ms}}$$

Remark: the seek time dominates the access time — this is exactly why disk scheduling algorithms (SSTF, SCAN, …) try to minimise head movement.

12.2 Solutions — I practise

SOLUTION **Exercise 5 — Comparison of I/O Transfer Mechanisms**

Criterion	Programmed I/O	Interrupt-driven I/O	DMA
CPU involvement during transfer	Total: CPU polls the status register and moves every byte itself	Partial: CPU is interrupted for each byte/small unit, free between interrupts	Minimal: CPU only starts and ends the transfer; the DMA controller moves whole blocks
Speed / Efficiency	Slowest; CPU wasted in busy waiting	Better; CPU does useful work between interrupts	Fastest and most efficient for large block transfers
Typical example device	Keyboard, simple sensors	Keyboard, mouse, serial port	Hard disk, network card, sound card
Complexity of implementation	Very simple (simple loop in the driver)	Moderate (interrupt handler + vector)	High (needs a DMA controller and bus arbitration)

SOLUTION **Exercise 6 — File Access Methods**

1. Reading all records from beginning to end: sequential access. The records are read one after another in order; sequential access is the simplest and fastest method for

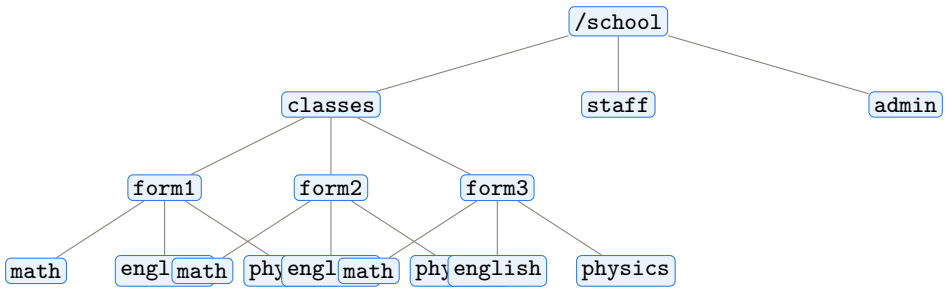
a full scan because the read head (or file pointer) simply advances, and the OS can prefetch the next blocks.

2. Jumping directly to record 500 and reading around it: direct (relative) access. With fixed-length records, the position of record n is computed directly: offset $= (n - 1) \times$ record length. For record 500, the OS positions the pointer at byte $499 \times L$ without reading the 499 previous records. (Indexed sequential access would also work, but direct access is the most natural for fixed-length records with known numbers.)
3. Inserting a new record at the end: sequential access (position the file pointer at end-of-file and append). Appending at the end is a sequential operation; it does not disturb the existing order and is $O(1)$ — no other record needs to be moved.

Conclusion: a file with fixed-length records supporting both sequential and direct access (a relative file) satisfies all three requirements efficiently.

SOLUTION **Exercise 7 — Directory Tree and Paths**

Directory tree:



Directory tree of the school system (subject directories contain the student files `john.txt`, `mary.txt`, …).

Under each subject directory (`math`, `english`, `physics`) we place the student files, e.g. `john.txt` and `mary.txt`.
Absolute path of `john.txt` in `math` for `form2`:

```
/school/classes/form2/math/john.txt
```

The path is absolute because it starts from the root `/` and lists every directory down to the file.

SOLUTION **Exercise 8 — File Allocation Methods**

The file has 10 blocks; we want block number 8 (the directory entry is already in memory).

1. Contiguous allocation:
   - Disk accesses: 1. The directory entry gives the address b of the first block and the length; block 8 is simply at address $b + 7$, read in a single access.
   - Advantage: fastest possible — both sequential and direct access are immediate.
   - Drawback: for a 10-block file we must find 10 consecutive free blocks; the file cannot grow beyond the reserved run, and external fragmentation appears over time.
2. Linked allocation:
   - Disk accesses: 8. Each block contains a pointer to the next one; to reach block 8 we must read blocks 1, 2, … , 8 in turn, following the chain — 8 disk reads.
   - Advantage: no external fragmentation; the file grows easily by appending any free block.
   - Drawback: direct access is impossible — reaching block 8 costs 8 reads, and one lost pointer corrupts the rest of the file.
3. Indexed allocation (single index block):
   - Disk accesses: 2. First read the index block (which contains the 10 block addresses), then read block 8 directly: $1 + 1 = 2$ accesses.
   - Advantage: supports direct access with few reads and no external fragmentation.
   - Drawback: the index block consumes one extra disk block (overhead of $1/10 = 10\%$ for this small file) and one extra access.

Method	Accesses for block 8	Advantage	Drawback
Contiguous	1	Fastest, direct access	Needs 10 consecutive blocks; external fragmentation
Linked	8	No fragmentation, easy growth	Slow direct access; fragile chain
Indexed	2	Direct access, no fragmentation	Index block overhead

12.3 Solutions — I prepare for the GCE

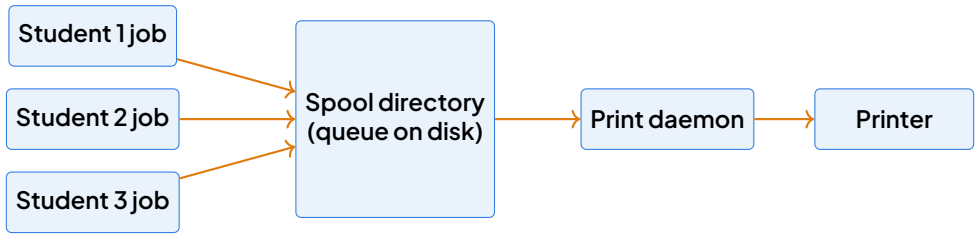
SOLUTION **Exercise 9 — Structured Question: Device Management and Spooling (15 marks)**

(a) Definitions (3 marks — 1 mark each)

- Device driver: a software module of the OS that controls a specific I/O device; it translates generic OS requests (read, write) into the device-specific commands of the hardware controller.
- Buffering: temporary storage of data in a memory area (buffer) during a transfer, to absorb the speed difference between the CPU and the I/O device.
- Spooling: technique in which jobs for a non-shareable device (e.g. a printer) are queued on disk and sent one by one to the device by a daemon, allowing several processes to submit jobs simultaneously.

(b) How spooling shares one printer between three students (6 marks)
Expected elements: spool directory on disk, queue, print daemon, non-blocking behaviour, order of service, mutual exclusion.

1. Each student's application "prints" by writing its job (the document plus control information: owner, number of copies, priority) as a file into the spool directory on disk. Writing to disk is fast, so all three students finish "printing" almost immediately and continue their work — they are not blocked waiting for the slow printer. (2 marks)
2. The jobs form a queue (usually FIFO). The printer is never accessed directly by the students' processes, so there is no conflict: only one process ever talks to the printer. (2 marks)
3. A system process called the print daemon takes the jobs from the queue one at a time, opens the printer, sends the data, and only then removes the job from the spool directory. The single printer is thus shared fairly and serially among the three students. (2 marks)



Spooling: jobs are queued on disk; the daemon feeds the printer one job at a time.

(c) Recovery after a paper jam (6 marks — any coherent sequence of steps, 1–2 marks each)

1. Detect and suspend: the printer signals an error (interrupt/status flag); the daemon suspends the current job (student 2) and marks it as "interrupted", recording how

much was already printed.

2. Do not lose the queue: the job files remain safely in the spool directory on disk, so student 3's job and the unfinished part of student 2's job are not lost.
3. Notify: the daemon informs the administrator (and/or the user) of the jam and waits for the fault to be cleared.
4. Resume fairly: after repair, the daemon restarts student 2's job (from the beginning or from the last correctly printed page), before starting student 3's job, so the original FIFO order and fairness are preserved.
5. Log the event: the incident is written to the system log for later diagnosis.

Examiner's expectations: the candidate must show that the spool area on disk provides persistence (jobs survive the fault) and that the daemon guarantees order and fairness; simply saying "restart the printer" earns little credit.

SOLUTION **Exercise 10 — Scenario: Disk Scheduling and File Corruption (15 marks)**

Queue: 82, 170, 43, 140, 24, 16, 190. Start at cylinder 50, moving towards higher cylinders.
(a) Total head movement (9 marks — 3 per algorithm: 1 for the order, 2 for the calculation)
(i) FCFS — requests served in arrival order:

$$50 \rightarrow 82 \rightarrow 170 \rightarrow 43 \rightarrow 140 \rightarrow 24 \rightarrow 16 \rightarrow 190$$

$$M = |82 - 50| + |170 - 82| + |43 - 170| + |140 - 43| + |24 - 140| + |16 - 24| + |190 - 16|$$
$$= 32 + 88 + 127 + 97 + 116 + 8 + 174 = \boxed{642 \text{ cylinders}}$$

(ii) SSTF — always serve the closest pending request:

- From 50: closest is 43 (distance 7).
- From 43: closest is 24 (19).
- From 24: closest is 16 (8).
- From 16: closest is 82 (66).
- From 82: closest is 140 (58).
- From 140: closest is 170 (30).
- From 170: closest is 190 (20).

Order: 50 → 43 → 24 → 16 → 82 → 140 → 170 → 190

$$M = 7 + 19 + 8 + 66 + 58 + 30 + 20 = \boxed{208 \text{ cylinders}}$$

(iii) SCAN (elevator) — moving upwards first, serve all requests up to the end of the disk, then reverse. Assuming the disk has 200 cylinders (0–199):

- Upwards from 50: 82, 140, 170, 190, then continue to cylinder 199 (end of disk).
- Reverse direction: 43, 24, 16.

Order: 50 → 82 → 140 → 170 → 190 → 199 → 43 → 24 → 16

$$M = (199 - 50) + (199 - 16) = 149 + 183 = \boxed{332 \text{ cylinders}}$$

(Equivalently: 32 + 58 + 30 + 20 + 9 + 156 + 19 + 8 = 332.)
Comment: SSTF gives the smallest movement here (208) but can starve far-away requests; SCAN (332) is fairer; FCFS (642) is simple but inefficient.
(b) Why the copied file is corrupted (3 marks) A file copy is not atomic: the OS writes the data blocks one after another and updates the metadata (directory entry, file size, allocation table/inode, free-block map) separately, often using cached writes. When the power cut occurs mid-copy:

- only some data blocks have been physically written, so the file content is incomplete or mixed with old data (1 mark);
- the metadata may be inconsistent with the data: e.g. the allocation table may record blocks as belonging to the file that were never written, or the size field may not match the blocks actually copied (1 mark);
- the free-block map may be wrong, leaving blocks marked both free and allocated (1 mark).

The file system is left in an inconsistent state, hence the corruption.
(c) Two protective mechanisms / good practices (3 marks — 1.5 each, any two)

- Journalling file system (e.g. ext3/ext4, NTFS): every change is first written to a log (journal); after a crash the OS replays or rolls back the journal to restore a consistent state.
- Safe removal / flushing: the user clicks "Eject/Safely remove" so the OS flushes all cached writes to the USB drive before it is unplugged; never remove the drive during a copy.
- Copy-then-rename (atomic update): write to a temporary file and rename it only when the copy is complete, so the original is never half-overwritten.
- Use of a UPS (uninterruptible power supply) so the machine can finish or abort cleanly during a power cut.

Examiner's expectations: precise vocabulary (metadata, journal, cache flush, consistency) is required for full marks; vague answers such as "use antivirus" earn nothing.

SOLUTION **Exercise 11 — Essay Question: OS Security and File Permissions (15 marks)**

(a) Interpretation of `rwxr-x---` for `grades.txt` (4 marks)
The 9 permission bits split into three triplets:

Category	Bits	Rights
Owner (user)	`rwx`	read, write and execute: full control
Group	`r-x`	read and execute, but no write
Others	`---`	no right at all (cannot read, write or execute)

(1 mark for the triplet decomposition, 1 mark per category correctly interpreted.)
(b) The `chmod` command (3 marks)
Requirements: owner `admin`: read+write; group `teachers`: read+write; others (students): read only. That is `rw- rw- r--` = 664:

- Octal notation: `chmod 664 grades.txt`
- Symbolic notation: `chmod u=rw,g=rw,o=r grades.txt`

(2 marks for the correct rights, 1 mark for correct syntax. Note: since students are neither the owner nor in group `teachers`, they fall in "others" and get read-only — this reasoning must appear.)
(c) Authentication vs authorisation (4 marks)

- Authentication verifies who you are — it checks the identity of a user or process before access to the system. Example: login with username and password, PIN, fingerprint/face recognition, two-factor authentication. (2 marks)
- Authorisation determines what you are allowed to do once authenticated — it checks rights on objects. Example: file permissions (`rwx`), access control lists, administrator vs standard accounts. (2 marks)

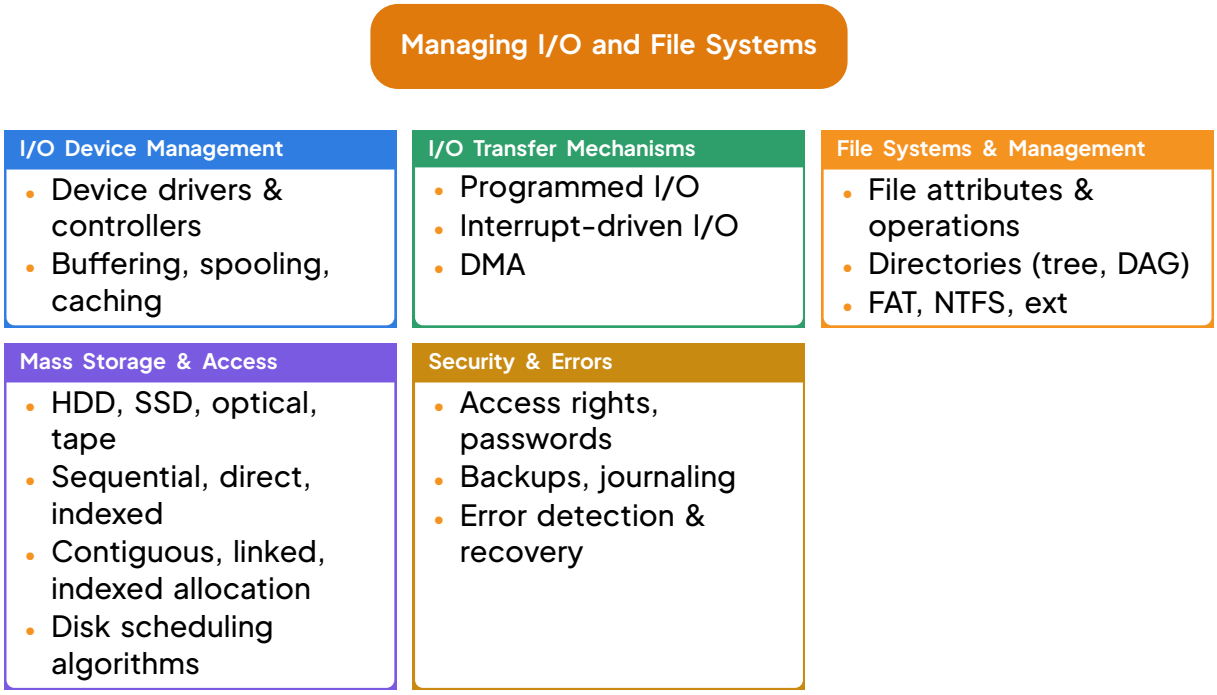
In short: authentication happens first ("prove your identity"); authorisation happens on every access ("are you allowed this operation on this object?").
(d) Two OS-level protections for the password file (4 marks — 2 each)

1. File permissions / ownership: the password file (e.g. `/etc/shadow`) is owned by `root` with permissions such as `rw-------` (600); a malicious program running as an ordinary user simply receives "permission denied" from the kernel, which checks rights on every `open()`.
2. Privileged execution mode (user/kernel mode) and least privilege: ordinary programs run in user mode and cannot directly modify system resources; only processes with administrator/root privileges may touch system files, and modern OSs require an explicit privilege elevation (UAC prompt, `sudo` with password) which the malicious program cannot obtain silently.

(Also acceptable: mandatory access control such as SELinux, read-only mounting of system partitions, integrity checking.)

Examiner's expectations: precise separation of the two concepts in (c), exact bit-by-bit reading in (a), and mechanisms located at the OS level (not antivirus or user vigilance) in (d).

13 SUMMARY SHEET



Mind map of the chapter: the five lessons around input/output management and file systems.

FICHE BILAN **L'essentiel à connaître pour l'examen**

1. Key definitions

- **I/O device**: any peripheral (keyboard, printer, disk, network card) through which the computer exchanges data with the outside world. Devices are classified as block devices (disks, transfer fixed-size blocks, e.g. 4 KiB) or character devices (keyboard, printer, transfer byte streams).
- **Device controller**: the electronic interface between a device and the system bus; it contains registers (data, status, control) that the CPU reads/writes via I/O instructions or memory-mapped I/O.
- **Device driver**: the OS kernel module that hides hardware specifics and presents a uniform interface (open, read, write, close, ioctl) to the rest of the system.
- **Interrupt**: a hardware signal sent by a device/controller to the CPU announcing an event (e.g. transfer complete, error). The CPU suspends the current process, saves context, and executes the corresponding interrupt handler (ISR).
- **DMA** (Direct Memory Access): a dedicated controller transfers whole blocks be-

tween device and main memory without CPU intervention; the CPU is interrupted only once per block (or at the end of a multi-block transfer).

- **Buffering**: a memory area used to temporarily hold data between a fast and a slow component to smooth speed mismatches (e.g. keyboard input buffer, disk cache).
- **Spooling** (Simultaneous Peripheral Operations On-Line): queuing entire jobs (e.g. print jobs) on disk so that a slow dedicated device can be shared among processes without blocking them.
- **Caching**: keeping copies of frequently accessed data in faster storage (e.g. disk cache in RAM) to reduce access latency.
- **File**: a named logical collection of related data stored on secondary storage; the OS views it as a sequence of bytes or records.
- **Directory**: a special file that maps file names to metadata (attributes, location on disk). A **path** specifies the location in the directory hierarchy (absolute from root, or relative to the current directory).
- **File system**: the OS component that organises, names, stores, protects, and retrieves files (examples: FAT32, NTFS, ext4).

2. I/O transfer mechanisms (know how to compare them)

Mechanism	Principle	CPU involvement
Programmed I/O (Polling)	CPU repeatedly reads the device status register until the device is ready, then transfers one word/byte.	Total: CPU wastes cycles in busy waiting; simple but inefficient for slow devices.
Interrupt-driven I/O	CPU initiates transfer, then switches to another process; device interrupts when ready.	Partial: CPU does useful work between interrupts; still one interrupt per word/byte.
DMA	CPU programs DMA controller (source, destination, count); DMA moves whole block directly to/from memory; one interrupt at end.	Minimal: CPU only sets up and handles completion interrupt; ideal for high-speed block devices.

3. File management essentials

- **File attributes**: name, type, size, location (disk blocks), owner, protection (access rights), timestamps (creation, last access, last modification).
- **File operations**: create, open (returns a file descriptor/handle), read, write, seek/reposition (move file pointer), delete, close, rename, truncate.

- **Directory structures**:
  - Single-level: one directory for all users (name clashes).
  - Two-level: one directory per user (no sharing).
  - Tree-structured (standard): root directory, subdirectories, files; absolute path from root (e.g. `/home/student/notes.txt`), relative path from current directory.
  - Acyclic-graph (DAG): allows shared files/directories via links (hard links or symbolic links); requires reference counting or garbage collection.
- **File system types**: FAT32 (simple, wide compatibility, no journaling, 4 GiB file limit), NTFS (journaling, permissions, encryption, large volumes), ext4 (Linux standard, journaling, extents, large files).
- **Access methods** (logical view):
  - Sequential: records processed in order (tape, log files); read next, write next, reset.
  - Direct/Random: any record accessed by its address/key (disk files); read n, write n, position to n.
  - Indexed: an index (e.g. B-tree, ISAM) maps keys to record addresses; supports fast search by key (databases).
- **Allocation methods** (physical placement on disk):
  - Contiguous: file occupies consecutive blocks; fast sequential and direct access; suffers external fragmentation; difficult to extend.
  - Linked: each block points to the next; no external fragmentation; slow direct access (must traverse); unreliable if a pointer is lost.
  - Indexed: an index block (or i-node) holds all block pointers; supports direct access; index block size limits file size (solved by multi-level indexing, e.g. UNIX inodes with direct, indirect, double-indirect pointers).

4. Mass storage — orders of magnitude, formulas, and disk scheduling

- **Disk access time** = seek time (moving head to track) + rotational latency (waiting for sector) + transfer time (reading/writing data).
- **Average rotational latency** = $\frac{1}{2} \times \frac{60}{N}$ seconds, where N is rotation speed in rpm (e.g. 7200 rpm → 4.17 ms).
- **Transfer time** = $\frac{\text{bytes transferred}}{\text{transfer rate}}$; transfer rate = $\frac{\text{bytes per track} \times N}{60}$.
- **Capacity** = surfaces × tracks per surface × sectors per track × bytes per sector.

- **Storage hierarchy** (fastest to slowest, cost per byte decreasing): registers, cache (L1/L2/L3), RAM, SSD (NAND flash), HDD (magnetic disk), optical disc (DVD/Blu-ray), magnetic tape (archival).
- **Disk scheduling algorithms** (minimise seek time):
  - FCFS (First-Come-First-Served): fair, but high average seek.
  - SSTF (Shortest Seek Time First): selects request with minimum seek from current head; may starve distant requests.
  - SCAN (Elevator): head moves in one direction, serving requests, then reverses; no starvation.
  - C-SCAN (Circular SCAN): head moves in one direction only, returns to start without serving; more uniform wait times.
  - LOOK / C-LOOK: variants that reverse direction at the last request instead of the physical end.

5. Security and error management

- **Protection mechanisms**:
  - User authentication: passwords, biometrics, two-factor authentication.
  - Access rights: per user/group (read, write, execute) on files/directories; represented as `rwx` triplets (owner, group, others) in UNIX-like systems; ACLs (Access Control Lists) for finer granularity.
  - Encryption: at rest (full-disk encryption, e.g. BitLocker, LUKS) and in transit (TLS).
- **Error management**:
  - Detection: parity bits, checksums, CRC (Cyclic Redundancy Check) on disk blocks and network frames.
  - Correction: ECC (Error-Correcting Code) memory, RAID (Redundant Array of Independent Disks) levels 1, 5, 6; retransmission for network.
  - Recovery: backups (full, incremental, differential), journaling file systems (ext4, NTFS) that log metadata changes to quickly restore consistency after a crash; fsck/chkdsk utilities.
- **Backup types**:
  - Full: all selected files.
  - Incremental: only files changed since the last backup (of any type); fastest backup, slowest restore (need full + all incrementals).
  - Differential: files changed since the last full backup; larger than incremental but restore needs only full + last differential.

6. Common mistakes to avoid

- Confusing interrupt-driven I/O with DMA: in DMA the CPU is interrupted only once per block (or per transfer), not once per byte/word.
- Saying spooling and buffering are the same: spooling queues whole jobs on disk for a dedicated device (printer), buffering smooths speed differences between any two components.
- Mixing up access methods (how records are read logically) with allocation methods (how blocks are placed physically on disk).
- Forgetting that contiguous allocation causes external fragmentation, while indexed allocation costs extra blocks for the index (but no external fragmentation).
- Writing a relative path where an absolute path (starting from the root) is required.
- Omitting units in disk access time calculations (seek time in ms, rotational latency in ms, transfer time in ms).
- Assuming SSTF is optimal; it minimises individual seek but not total head movement (SCAN/LOOK are better for overall throughput).

7. GCE tips

- When asked to "explain how data reaches memory from disk", structure the answer in three stages: programmed I/O → interrupt-driven → DMA, stating the CPU's role at each stage.
- In disk-scheduling or access-time problems, always show the formula, substitute values with units, and give the final answer with its unit (ms, MB, GB).
- For essay questions on file systems, always illustrate with a concrete example (FAT32 on a USB stick, ext4 on a Linux server in a Cameroonian cybercafé) and mention at least one advantage and one limitation.
- Learn to draw and label a simple tree-structured directory and to trace a path from the root: this scores easy marks.
- Be ready to compare allocation methods in a table (contiguous vs linked vs indexed) with criteria: sequential access, direct access, fragmentation, ease of extension.
- Know the difference between hard links (same inode, cannot cross file systems, cannot link directories) and symbolic links (separate inode, can cross file systems, can link directories).

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+